Artificial Intelligence (AI) Agents

Companion Guide v1.0

CIS Critical Security Controls v8.1

April 2026

CIS Center for Internet Security®

CIS Controls®

Acknowledgments

The Center for Internet Security, Inc. (CIS®) would like to thank the many security experts who volunteer their time and talent to support the CIS Critical Security Controls® (CIS Controls®) and other CIS work. CIS products represent the effort of a veritable army of volunteers from across the industry, generously giving their time and talent in the name of a more secure online experience for everyone.

As a nonprofit organization driven by its volunteers, we are always in the process of looking for new topics and assistance in creating cybersecurity guidance. If you are interested in volunteering and/or have questions, comments, or have identified ways to improve this guide, please email us at: controlsinfo@cisecurity.org.

All references to tools or other products in this guide are provided for informational purposes only, and do not represent the endorsement by CIS of any particular company, product, or technology.

Principal Author

Jonathan Sander, Astrix

Editors

Andrew Dannenberger, CIS

Robin Regnier, CIS

Thomas Sager, CIS

Valecia Stocchetti, CIS

Contributors

Abhishek Iyer, Cybersecurity Leader

Christopher Misra, University of Massachusetts

Jack Zaldivar Jr., Databricks

Jeremy Pelegrin, University of Massachusetts

Michael Laing, Loblaw Companies Limited

Contents

Executive Summary

Artificial Intelligence (AI) agents represent a rapidly emerging class of systems that blend Large Language Models (LLMs) with orchestration logic, tool execution, data retrieval, and automated decision-making. Unlike stand-alone models, AI agents operate across multiple layers of an enterprise's environment, interacting with internal services, external APIs, sensitive data, and user workflows. This expanded operational footprint introduces new risks, including unauthorized actions, data leakage, and unintended system changes, that require security considerations beyond traditional model-centric safeguards.

CIS Controls v8.1 offers foundational, prioritized cybersecurity best practices designed to help enterprises mitigate the most prevalent threats. However, applying these Controls directly to AI agents requires interpreting them through the lens of autonomous and semi-autonomous system behavior. Agent architectures often extend across identity layers, endpoint execution environments, knowledge stores, integration pipelines, and operational monitoring systems, meaning the Controls must be mapped to a broader attack surface than conventional software.

This guide provides practical, actionable guidance for applying CIS Controls v8.1 to the agent layer specifically, the layer where planning, reasoning, tool invocation, and multi-step workflows occur. By aligning established security practices with the unique operational characteristics of AI agents, enterprises can strengthen their security posture while maintaining the flexibility and innovation that AI systems enable. Each section interprets relevant CIS Safeguards in the context of agent behavior, providing clarity on where traditional controls still apply and where new patterns must be considered.

As enterprises adopt AI agents to streamline processes, enhance decision-making, and automate complex tasks, ensuring secure design and operation becomes essential. This guide aims to bridge the gap between standard cybersecurity frameworks and emerging AI agent architectures, helping teams implement controls that reduce risk while supporting responsible, reliable, and resilient use of AI technologies.

Scope

This guide applies to systems that implement AI agents or multi-agent workflows, including agents that:

- Perform task decomposition or planning
- Execute internal or external tools or Application Programming Interfaces (APIs)
- Interact with code interpreters, browsers, or business data systems
- Maintain short-term or long-term memory
- Access vector databases, key-value stores, or structured data sources
- Run on hosted agent platforms, cloud workloads, or event-driven systems
- Use the Model Context Protocol (MCP) for structured tool exposure

This guide assumes that all underlying model behavior is governed by the AI LLM Companion Guide and does not duplicate model-level best practices in that guide.

Multimodal Capabilities and Non-Text Modalities

AI Agents frequently utilize multimodal models to "see" screens, process documents, or analyze audio. Enterprises must recognize that non-text inputs function as prompts. Just as a text prompt can contain malicious instructions, an image or audio file can contain hidden, embedded, or adversarial patterns ("Multimodal Injections") that manipulate the agent's reasoning and subsequent actions.

This guide governs non-text modalities specifically as input vectors for agent reasoning and action.

- **In Scope:** Risks related to "Multimodal Injection," where an image (e.g., a screenshot containing hidden text) or audio file is used to hijack the agent's context, bypass guardrails, or trigger unauthorized tool use.
- **Out of Scope:** Risks related to the processing mechanics of these files (e.g., buffer overflows in image parsers) or domain-specific concerns such as biometric privacy or deepfake generation.

Where non-text capabilities are enabled, enterprises must apply the relevant Safeguards in Control 9 (Email and Web Browser Protections), Control 15 (Service Provider Management), Control 16 (Application Software Security), and Control 18 (Penetration Testing) to these inputs with the same rigor applied to text.

Retrieval-Augmented Generation (RAG) and Scope Boundaries

Agents often use Retrieval-Augmented Generation (RAG) mechanisms to provide long-term memory or access to knowledge bases. While the storage infrastructure is out of scope, the integrity of the agent's memory is in scope. The agent's reliance on retrieved context is in scope because it directly influences reasoning, tool selection, and actions.

A compromised RAG index does not just leak data; it can alter an agent's decision logic, such as by retrieving a fake policy to bypass a guardrail. Therefore, this guide treats RAG not as a storage component, but as a behavioral dependency. Controls focus on preventing "memory poisoning" and ensuring that agents validate and do not blindly trust retrieved context.

When RAG is present, assess the retrieval layer as a risk surface across confidentiality, integrity, and availability:

- **Confidentiality:** Retrieval can expose sensitive information through agent outputs, summaries, or tool calls. The agent may surface data beyond the requester's authorization or propagate sensitive content into downstream systems such as tickets, emails, logs, or chat transcripts.
- **Integrity:** Poisoned or manipulated content can change agent decisions and actions. This includes malicious entries, altered documents, or injected instructions that cause the agent to select unsafe tools, misuse privileges, or produce incorrect outcomes.
- **Availability:** Retrieval outages, index corruption, or stale data can break workflows or push the agent into degraded behavior. Poorly designed fallbacks, such as continuing without verification or broadening sources when retrieval fails, can create unsafe defaults.

Controls in this guide that address input validation, memory integrity, and tool governance apply to retrieved content regardless of where it is stored.

Topics Not Covered

The guide does not address:

- Model training, fine-tuning, or dataset curation
- Governance of non-AI automation systems, except where they are exposed as tools
- Evaluation of generative media authenticity
- Supply-chain validation of third-party model artifacts beyond what is covered in other CIS documents

These areas require additional controls and frameworks outside the scope of this guide. Guidance on these areas can be found in the AI LLM Companion Guide.

Methodology

This guide follows the structure and intent of CIS Controls v8.1 and is designed to be read alongside the primary CIS Critical Security Controls v8.1, the CIS Controls Cloud Companion Guide, and the AI LLM Companion Guide. It does not introduce new top-level Controls or Safeguards; instead, it interprets each existing Control in the context of AI agents, systems that perform multi-step reasoning, invoke tools, interact with data sources, maintain memory, and take actions within enterprise environments.

For each CIS Control (1–18), this guide provides:

- **AI Agent Applicability:** A short explanation of how the Control applies to agentic systems, including orchestration logic, tool usage, memory and retrieval behavior, and autonomous or semi-autonomous actions.
- **Safeguards:** Guidance that adapts the CIS Safeguards to agent-specific contexts. These are the primary actionable elements that demonstrate how AI agents apply to the CIS Safeguards.
- **Model Hosting and Deployment Considerations:** A brief discussion of how each Safeguard's AI Agent Applicability applies across different deployment environments (cloud-hosted, on-premises/private, endpoint/edge) and different patterns of runtime ownership and control (provider-managed, enterprise-managed, local/embedded). These dimensions determine where guidance must be enforced and how responsibilities are shared.
- **Additional AI Agent Considerations:** Additional nuances, edge cases, or contextual guidance that helps enterprises tailor the Safeguards to specific agent architectures or operational patterns.

This guide also maintains the following design principles:

- **Layering with Related Companion Guides**
  - The AI LLM Companion Guide provides the model-level expectations for how Large Language Models (LLMs) and Small Language Models (SLMs) handle text and context; this document builds directly on that foundation.
  - The MCP Companion Guide provides guidance where tools or capabilities are exposed or governed through the Model Context Protocol (MCP).
- **Agent Life Cycle Coverage**
  This guide addresses the full life cycle of agentic systems: design, configuration, deployment, memory and retrieval management, tool integration, monitoring, and retirement.
- **Risk-Based Tailoring**
  Not all agents require the same rigor. The guidance presented here should be applied proportionally to the risk posed by the agent's autonomy level, tool surface, data access, data sensitivity, and business impact.

Enterprises should read each Control section in light of:

- Their selected Implementation Group(s)
- Their deployment environment (cloud, on-premises/private, endpoint/edge)
- Their runtime ownership and control model (provider-managed, enterprise-managed, local/embedded)
- The presence or absence of memory, tool invocation, and retrieval/RAG capabilities

How to Use This Guide

Implementation Groups (IG1, IG2, IG3) continue to guide prioritization as with all preceding CIS Controls guides. Agents with broad tool access, high autonomy, or significant operational impact may require safeguards from higher Implementation Groups regardless of the enterprise's general IG classification.

This guide assumes the enterprise has implemented foundational CIS Controls appropriate to its operating environment. Specifically, it assumes enterprises have:

- Adopted the guidance in the AI LLM Companion Guide for all model usage
- Adopted the guidance in the MCP Companion Guide for tool exposure, authentication, and authorization
- A secure software development life cycle aligned with CIS Control 16
- Enterprise Identity and Access Management (IAM), logging, monitoring, and incident response processes capable of supporting agent operations

AI agents must be treated as full software applications with additional risks introduced by autonomy, tool capabilities, and multi-step reasoning.

This guide extends CIS Controls v8.1 for agent-based architectures. It complements the CIS Controls Cloud Companion Guide for cloud-hosted deployments, the AI LLM Companion Guide for model-layer best practices, and the MCP Companion Guide for securing tool interfaces and structured agent–system integrations.

Key Concepts and Terminology

Agent Architecture

For clarity, this guide treats an AI agent as a multi-component application consisting of:

- **Interface Layer** – User interfaces, APIs, software development kits (SDKs), or message inputs
- **Orchestration and Planning Layer** – Routing, reasoning, step sequencing, and decision logic
- **Model Layer** – One or more Large Language Models or related models used within the agent
- **Tool and Action Layer** – APIs, business systems, code execution tools, browsers, and MCP-exposed capabilities
- **Memory and Knowledge Layer** – Short-term and long-term state, vector stores, and structured data sources
- **Evaluation and Observability Layer** – Logging, telemetry, evaluation pipelines, and red-team harnesses
- **Infrastructure Layer** – Compute environments hosting the agent, including on-premises, cloud, or managed agent platforms

This layered view ensures that security considerations apply appropriately to the components responsible for planning, accessing external resources, updating state, and performing operational actions.

Agent Deployment, Hosting, and Runtime Control

AI agents can be operated in different ways depending on where they run and who manages the runtime that performs their orchestration, memory handling, and tool interactions. Throughout this guide, two concepts are used to describe these differences: Deployment Environments and Runtime Ownership and Control. These concepts influence how controls apply across infrastructure, applications, and agent behavior.

Deployment Environments

Deployment environments describe where an agent's runtime executes. These three environments align with the assumptions and responsibility boundaries defined in *CIS Controls v8.1* and the *CIS Controls Cloud Companion Guide*.

- **Cloud-Hosted Environments:** Agents executed in public or hybrid cloud infrastructures, including managed compute services, serverless platforms, and cloud-hosted application runtimes. These environments apply cloud-specific policies for identity, network

segmentation, storage protections, and data governance.

- **On-Premises or Private Infrastructure:** Agents executed on enterprise-operated infrastructure such as private data centers, self-managed clusters, or dedicated compute platforms. Enterprises are responsible for the full security stack, including physical protections, network controls, storage security, and operational safeguards.
- **Endpoint or Edge Environments:** Agents embedded within client devices, local applications, or edge compute systems. These deployments rely on endpoint protections, local storage controls, device hardening, and data-loss prevention mechanisms appropriate to laptops, mobile devices, and edge hardware.

Runtime Ownership and Control

Runtime ownership describes who operates and governs the agent's orchestration loop, working memory, intermediate state, and tool-invocation logic. This concept parallels the model-hosting distinctions introduced in the AI LLM Companion Guide, but applies at the agent-runtime layer.

- **Provider-Managed Runtimes:** The agent's execution environment, state handling, intermediate memory, and orchestration logic are operated by a third-party provider. Enterprises configure agent behavior but do not operate or directly secure the underlying runtime.
- **Enterprise-Managed Runtimes:** The agent runtime is operated within infrastructure controlled by the enterprise. The enterprise manages orchestration code, memory stores, retrieval integrations, and tool interactions, even when model inference or certain services are externally hosted.
- **Local or Embedded Runtimes:** The agent runtime executes directly on endpoint or edge devices. Orchestration, intermediate reasoning state, and data handling occur locally, with optional access to remote model or storage services.

Applying Deployment Environments and Runtime Control

In practice, an agent's characteristics depend on both where it runs and who manages the runtime. Common patterns include:

- An enterprise-managed runtime deployed in a cloud compute environment (e.g., an agent built with an open-source framework and deployed on a managed container platform offered by a major cloud provider)
- An agent executed in a provider-managed runtime running in the provider's cloud environment (e.g., a fully managed "agent builder" service that performs orchestration and memory handling within the provider's infrastructure)
- An embedded runtime located on an endpoint device (e.g., a local assistant integrated into a workstation application and supported by remote model APIs)

These combinations appear throughout the Controls and determine how data protections, identity safeguards, tool authorizations, logging, and monitoring requirements apply. Each Control includes a section describing how the Safeguards apply to AI Agents, as well as how they should be interpreted across the different deployment environments and runtime-ownership models.

Relationship to LLM Hosting Types

Readers familiar with the AI LLM Companion Guide will note that it categorizes systems into three Model Hosting Types: Endpoint-Hosted, Enterprise-Hosted, and SaaS-Hosted.

Since AI agents introduce complex middleware (orchestration, memory, and tools) that may sit apart from the model itself, this document separates where the code runs (Deployment Environment) from who manages the logic (Runtime Ownership).

To align your controls, map your agent architecture as follows:

- If you are running SaaS-Hosted Models (as described in the AI LLM Companion Guide), these often correspond to provider-managed runtimes in this guide. However, this is only if the SaaS provider also manages the agent's framework (i.e., the orchestration logic, memory state).
- If you run your own agent framework (e.g., LangChain, AutoGen) on cloud infrastructure but call a SaaS model API, you have an enterprise-managed runtime (Agent) using a SaaS-hosted model (LLM).
- If you are running Endpoint-Hosted Models (using things like Ollama or in embedded systems like you find in OT scenarios), these tend to correspond directly to local or embedded agent runtimes.

Glossary

Adversarial Evaluation	Testing of AI systems using intentionally crafted inputs (prompts, documents, data) designed to elicit unsafe, unintended, incorrect, or policy-violating unexpected behavior.
Agent	A system that uses a model to plan, reason, select or sequence actions, invoke tools, retrieve information, maintain state or memory, and perform multi-step operations in pursuit of a task.
Agent Action Trace	A record of an agent's decisions, tool calls, memory updates, retrieval operations, and intermediate steps that explain how it arrived at a final result.
Agent Autonomy	The degree to which an agent can initiate or sequence actions without explicit step-by-step human direction, often constrained by guardrails or policy layers.
Agent Configuration	The set of prompts, policies, tool definitions, memory rules, retrieval parameters, routing logic, and environment settings that govern agent behavior.
Agent Loop	The iterative cycle in which an agent evaluates state, plans an action, invokes a tool or model, interprets results, updates memory, and repeats until a stopping condition is met.
Agent Misbehavior	Any agent action or decision that violates policy, performs unsafe operations, deviates from intended task boundaries, or results from corrupted memory, poisoned retrieval content, adversarial prompts, or misconfiguration.
Agent Runtime	The execution environment in which an agent operates, including its orchestration engine, tool interfaces, memory stores, retrieval stack, and model-integration layer.
Agent State	The combination of working memory, retrieved content, intermediate outputs, tool results, and context that influences the agent's next action.
AI-Bill of Materials (AI-BOM/ Model BOM)	A structured record of models, dependencies, datasets, and components (frameworks, tokenizers, tools) used in a system, including version and provenance information.
AI Red-Teaming	Targeted testing of AI systems by internal or external experts using adversarial techniques specific to models, prompts, agents, and tool-driven workflows.
Augmentation Store/Retrieval Corpus	The collection of documents, embeddings, or data sources used by retrieval-augmented systems to supply external context to models or agents.
Behavioral Drift	Unintended changes in model or agent behavior over time that are not explicitly intended and may introduce safety or reliability issues that tend to occur after updates, fine-tuning, prompt changes, or environmental shifts.
Chain-of-Thought/Intermediate Reasoning	Internal or external reasoning steps used by a model or agent to break down tasks. Enterprises may restrict logging or exposure based on sensitivity and policy.
Concentration Risk (AI)	The risk that over-reliance on a single model, provider, or tool ecosystem creates systemic security or resilience vulnerabilities.
Data Leakage	Exposure of sensitive information through prompts, agent outputs, logs, retrieval content, memory states, tool responses, or embeddings.
Data Poisoning	Manipulation of training, fine-tuning, memory entries, or retrieval data to embed malicious or harmful behaviors into a model, the model's outputs, or agents.
Delegation (Agent)	When an agent assigns part of a task to another agent or subsystem, often through messaging, tool calls, or workflow frameworks.
Embedding/Embedding Model	A numeric vector representation of content produced by a model for similarity search, retrieval, or clustering.
Enterprise-Hosted Model	A model deployed on infrastructure controlled by the enterprise (on-premises, private cloud, or private VPC).
Execution Sandbox	A restricted environment in which an agent may execute code or perform file or browser operations with limited privileges and isolation from sensitive systems.
Execution Tool	A tool that runs code, scripts, commands, or programs on behalf of an agent, typically within a sandbox or constrained environment.
Fine-Tuning	Additional training of a base model on task- or domain-specific data to adapt its behavior without full retraining from scratch.

Guardrail	Application or middleware logic that constrains or validates model and agent behavior, inputs, outputs, or tool actions, enforcing business rules and safety policies outside the model.
Implementation Group (IG)	Grouped IG1, IG2, and IG3, these are a way for enterprises to prioritize the implementation of the CIS Controls.
Jailbreak	An attempt to circumvent model safety controls or policies using crafted prompts or adversarial context.
Kill Switch	A control or mechanism that allows rapid disabling of a model, endpoint, tool capability, agent, or entire AI subsystem in response to an incident.
Long-Term Memory (Agent)	Persisted information, such as summaries, structured data, preferences, or historical context, which agents use across sessions or workflows.
Memory (Short-Term/Working)	Transient state maintained during an active agent session, including retrieved documents, intermediate reasoning, and tool outputs.
Model Card/System Card	Documentation describing a model's capabilities, limitations, risk factors, and intended uses.
Model Context/Context Window	The range of tokens (input and/or prior output) that the model can attend to when generating a response.
Model Extraction	Attempts to replicate or approximate a proprietary model's behavior (or underlying parameters) by querying it at scale and analyzing responses.
Model Hosting Type	The deployment category describing where and how a model runs: endpoint-hosted, enterprise-hosted, or SaaS-hosted.
Model Provenance	The origin, lineage, and transformation history of a model, including base model, fine-tuning datasets, and training processes.
Multi-Agent Workflow	A coordinated process where multiple agents exchange messages, delegate tasks, or share memory or tools to complete a larger goal.
Non-Text Modality	Inputs or outputs in forms other than text, such as images, audio, or video.
Policy Enforcement Layer (Agent)	Logic that evaluates proposed agent actions (e.g., tool calls or memory writes) before execution, enforcing constraints independent of the model.
Prompt	The input content provided to a model or agent, including instructions, questions, or data examples.
Prompt Injection	An attack where malicious instructions are embedded in content processed by an LLM to manipulate its behavior. Indirect prompt injection occurs when hostile instructions arrive through retrieved resources, tool outputs, or other external content rather than direct user input.
Recursion Loop (Agent)	An uncontrolled or unintended repetition of agent planning cycles that can lead to runaway behavior or resource misuse.
Retrieval-Augmented Generation (RAG)	An architecture where external data is retrieved (e.g., via embeddings and vector search) and then incorporated into the model or agent context as additional input.
Retrieval Pipeline	The end-to-end process by which an agent embeds queries, performs vector search, retrieves documents, filters results, and integrates them into memory or prompts.
Retrieved Context	The content selected by retrieval mechanisms and used to guide model or agent behavior.
Runtime Ownership and Control	The entity responsible for operating and securing the environment in which an agent executes (e.g., provider-managed, enterprise-managed, or local/embedded).
SaaS-Hosted Model	A model operated by a third-party provider and exposed over API/SDK or web interface, where runtime and infrastructure are managed by the provider.
Scratchpad	A temporary, observable memory buffer where an AI agent records its intermediate reasoning steps, plans, and tool outputs before generating a final response. Unlike the hidden internal state of a model, the scratchpad is often visible in logs (e.g., "Chain-of-Thought"), making it a critical surface for monitoring agent intent and detecting potential jailbreak attempts or logic errors during execution.
Secure Web Gateway (SWG)	A control that inspects and governs web traffic, often providing URL filtering, content inspection, and DLP.

Shadow AI	Unapproved or unmanaged AI tools, accounts, agents, or services within an enterprise, such as personal accounts on public AI services or unsanctioned model deployments.
Small Language Model (SLM)	A smaller, more resource-efficient language model, typically used for constrained devices, specific tasks, or cost-sensitive deployments.
System Prompt	A privileged, often hidden, instruction block that sets a model's or agent's overall behavior, tone, or policy.
Tool (Model-Level Tool/Function Call)	An external capability an agent invokes, such as APIs, databases, execution engines, browsers, file handlers, or custom actions.
Tool Invocation	The process by which an agent selects a tool, constructs parameters, executes the action, and interprets results.
Tool Surface	The set of external capabilities available to an agent; a core part of the agent's attack surface.
User Prompt	Input provided by an end user or calling application to request an action from a model or agent.
Vector Store/Vector Database	A storage system optimized for similarity search over embeddings, typically used to implement retrieval in RAG systems.
Vector-Store Poisoning	The intentional injection of malicious or manipulated data into a vector database to corrupt similarity search results, designed to alter the retrieved context and compromise the outputs of a RAG-enabled agent.

Control 1: Inventory and Control of Enterprise Assets

Actively manage (inventory, track, and correct) all enterprise assets (end-user devices, including portable and mobile; network devices; non-computing/Internet of Things (IoT) devices; and servers) connected to the infrastructure physically, virtually, remotely, and those within cloud environments, to accurately know the totality of assets that need to be monitored and protected within the enterprise. This will also support identifying unauthorized and unmanaged assets to remove or remediate.

AI Agent Applicability

AI agents introduce new asset types that must be inventoried and governed. These include agent runtimes, orchestration components, tool interfaces, memory and state systems, model integrations, and retrieval pipelines. Some agents are long-lived services; others are instantiated dynamically in cloud workloads, serverless functions, or endpoint devices. Without proper inventory, enterprises cannot enforce access controls, data protections, or operational controls across the agent ecosystem.

Agent Asset Types

AI agents consist of multiple components, each representing an asset that may require individual tracking:

- **Agent runtimes** – the execution environment performing orchestration, planning, memory handling, and tool invocation
- **Agent configurations** – prompts, policies, tool lists, routing logic, and operational parameters
- **Tool interfaces** – API endpoints, MCP tools, code execution environments, browser tools, and other capabilities callable by the agent
- **Memory systems** – working memory, long-term memory, vector databases, and other state stores
- **Retrieval and embedding components** – vector pipelines, indexing jobs, embedding services, and document loaders
- **Dependent services** – external APIs, internal business systems, file stores, or execution sandboxes
- **Model integrations** – LLM or SLM endpoints referenced by the agent runtime

Each of these components must be discoverable, tracked, and evaluated for authorized use.

Safeguards

Control 1: Inventory and Control of Enterprise Assets

Safeguard	Title	Description	IG1	IG2	IG3	AI Agent Applicability
1.1	**Establish and Maintain Detailed Enterprise Asset Inventory**	Establish and maintain an accurate, detailed, and up-to-date inventory of all enterprise assets with the potential to store or process data, to include: end-user devices (including portable and mobile), network devices, non-computing/IoT devices, and servers. Ensure the inventory records the network address (if static), hardware address, machine name, enterprise asset owner, department for each asset, and whether the asset has been approved to connect to the network. For mobile end-user devices, MDM type tools can support this process, where appropriate. This inventory includes assets connected to the infrastructure physically, virtually, remotely, and those within cloud environments. Additionally, it includes assets that are regularly connected to the enterprise's network infrastructure, even if they are not under control of the enterprise. Review and update the inventory of all enterprise assets bi-annually, or more frequently.	●	●	●	Establish and maintain a detailed inventory of all agent assets, covering cloud-hosted platforms, on-premises orchestration servers, and embedded edge runtimes. The inventory must record the business owner responsible for the agent's actions, its approved connectivity paths to internal data stores (e.g., vector databases (DBs)) and external SaaS models (e.g., OpenAI, Anthropic), and the specific tools it is authorized to invoke. This includes tracking ephemeral agent workloads (serverless functions) and verifying that all agent instances map back to an authorized deployment pipeline and identity. Agents function as autonomous identities that bridge critical internal systems with external inference providers; without a comprehensive inventory that maps ownership, connectivity, and tool authorization, enterprises cannot effectively monitor for "shadow agents," manage the blast radius of a compromised agent, or enforce data egress policies across distributed and often ephemeral agent infrastructure.
1.2	**Address Unauthorized Assets**	Ensure that a process exists to address unauthorized assets on a weekly basis. The enterprise may choose to remove the asset from the network, deny the asset from connecting remotely to the network, or quarantine the asset.	●	●	●	Identify and quarantine unauthorized AI agent runtimes, including unapproved local agent tools on endpoints or rogue agent workloads in cloud environments. "Shadow agents" running on unmanaged assets can bypass data loss prevention (DLP) controls, exfiltrate sensitive data to external model providers, or execute unauthorized actions on internal networks.
1.3	**Utilize an Active Discovery Tool**	Utilize an active discovery tool to identify assets connected to the enterprise's network. Configure the active discovery tool to execute daily, or more frequently.		●	●	Use active discovery tools to detect AI agent orchestration services, vector database endpoints, MCP servers, and internal tool APIs exposed on the network. Agents often expose internal APIs or memory stores to facilitate tool use through MCP or other mechanisms; active discovery ensures these interfaces are visible and properly secured against unauthorized access or exploitation.
1.4	**Use Dynamic Host Configuration Protocol (DHCP) Logging to Update Enterprise Asset Inventory**	Use DHCP logging on all DHCP servers or Internet Protocol (IP) address management tools to update the enterprise's asset inventory. Review and use logs to update the enterprise's asset inventory weekly, or more frequently.		●	●	Scan all hosts discovered through DHCP for evidence of AI agent activity. Agents running in dynamic containerized environments may grab temporary IP addresses; DHCP logging provides a trail of where agent workloads have executed, aiding in asset tracking and incident investigation.
1.5	**Use a Passive Asset Discovery Tool**	Use a passive discovery tool to identify assets connected to the enterprise's network. Review and use scans to update the enterprise's asset inventory at least weekly, or more frequently.			●	Deploy passive network monitoring to detect traffic patterns indicative of agent communication, such as high-frequency API calls to model endpoints, MCP servers, or vector stores. Passive discovery helps identify "silent" or unmanaged agents that do not respond to active scans but are actively communicating with external tools or internal data systems.

Model Hosting and Deployment Considerations

Deployment Environments

- **Cloud-Hosted Agents:** Inventory must account for cloud-native components such as serverless runtimes, managed container services, managed vector databases, cloud APIs, and distributed data stores. Ensure cloud-specific assets are included in the enterprise Configuration Management Database (CMDB) or equivalent inventory systems.
- **On-Premises or Private Infrastructure Agents:** Inventory must include locally managed compute nodes, orchestration platforms, memory stores, retrieval pipelines, and any internal APIs or databases invoked by agents. Ensure manual or semi-automated discovery processes cover these internal assets.
- **Endpoint or Edge Agents:** Inventory must include locally deployed or embedded agent runtimes, agent-enabled applications, local tool connectors, and endpoint-resident configuration or memory artifacts. Ensure that device-level deployment tooling or endpoint management systems can register these agents.

Runtime Ownership and Control

- **Provider-Managed Runtimes:** Inventory must track which agents operate within vendor-managed execution environments. Document the provider, service boundaries, tool-integration mechanisms, configuration storage locations, and any state or logs stored under provider control.
- **Enterprise-Managed Runtimes:** Inventory must cover the enterprise-operated agent runtime, including code artifacts, orchestration logic, memory storage, retrieval jobs, and tool connectors running on enterprise infrastructure. Integrate inventory processes with deployment pipelines and internal hosting platforms.
- **Local or Embedded Runtimes:** Inventory must track which devices host embedded agent runtimes, which agents run locally, and what capabilities those local agents have. Ensure endpoint management systems or Mobile Device Management (MDM) tools can register and attribute embedded runtimes to specific devices and users.

Additional AI Agent Considerations

- Inventory should reflect the full life cycle of agents, including development, testing, staging, and production deployments.
- Multi-agent workflows may introduce implicit agents created dynamically; inventory processes must track these where feasible.
- Tool integrations may require tracking operational entitlements, such as API keys, service accounts, or MCP tool registrations.

Control 2: Inventory and Control of Software Assets

Actively manage (inventory, track, and correct) all software (operating systems and applications) on the network so that only authorized software is installed and can execute, and that unauthorized and unmanaged software is found and prevented from installation or execution.

AI Agent Applicability

AI agents rely on software stacks composed of orchestration frameworks, tool interfaces, planners, Software Development Kits (SDKs), retrieval components, memory systems, and model clients. These components evolve rapidly and may be installed across cloud services, on-premises infrastructure, development environments, or endpoint devices. Without visibility into the software dependencies supporting agents, enterprises cannot manage vulnerabilities, ensure secure configurations, or enforce policy across the agent's execution surface.

Software Components in the AI Agent Stack

AI agents depend on multiple software elements that must be inventoried:

- **Agent frameworks and orchestration libraries** – components responsible for sequencing steps, managing memory, routing tool calls, or coordinating multi-agent workflows
- **Tools and tool adapters** – local or remote executors, API clients, MCP tools, browser automation components, database connectors, and action handlers
- **Memory and retrieval libraries** – embedding services, vector-database clients, caching layers, and RAG pipelines
- **Model clients and SDKs** – libraries used to interact with LLM or SLM endpoints
- **Support dependencies** – logging frameworks, evaluation harnesses, dependency injection systems, and batch or event-processing components
- **Runtime plug-ins or extensions** – optional modules that enable specialized behaviors, such as code execution tools, visualization add-ons, or custom prompt interpreters
- **Local execution sandboxes** – interpreters, language runtimes, or safe-execution environments used when agents invoke code or scripts

These components collectively form the software asset layer for agent-based systems and must be tracked as part of the enterprise's standard software inventory.

Safeguards

Control 2: Inventory and Control of Software Assets

Safeguard	Title	Description	IG1	IG2	IG3	AI Agent Applicability
2.1	**Establish and Maintain a Software Inventory**	Establish and maintain a detailed inventory of all licensed software installed on enterprise assets. The software inventory must document the title, publisher, initial install/use date, and business purpose for each entry; where appropriate, include the Uniform Resource Locator (URL), app store(s), version(s), deployment mechanism, decommission date, and number of licenses. Review and update the software inventory bi-annually, or more frequently.	●	●	●	Maintain a comprehensive inventory of all AI agents' core software stacks and authorized tools, including LLM dependencies, orchestration frameworks, MCP clients/servers, and execution environments. The inventory must document business owners, license types, and versions for all components, treating tools, whether local or network-hosted, as independent software assets with distinct access privileges. Since agents rely on composite stacks where changes to a SaaS model or a local library can alter decision-making, rigorous tracking of these utilities is essential to manage supply chain risks and prevent the emergence of unmanaged "shadow software" that could be abused by a compromised agent.
2.2	**Ensure Authorized Software is Currently Supported**	Ensure that only currently supported software is designated as authorized in the software inventory for enterprise assets. If software is unsupported, yet necessary for the fulfillment of the enterprise's mission, document an exception detailing mitigating controls and residual risk acceptance. For any unsupported software without an exception documentation, designate as unauthorized. Review the software list to verify software support at least monthly, or more frequently.	●	●	●	Remove or update unsupported frameworks, outdated memory libraries, deprecated plug-ins, or obsolete SDKs. Ensure that agents do not rely on discontinued or vulnerable software components.
2.3	**Address Unauthorized Software**	Ensure that unauthorized software is either removed from use on enterprise assets or receives a documented exception. Review monthly, or more frequently.	●	●	●	Investigate unapproved tools, frameworks, plug-ins, or executors added to agent runtimes. Monitor for unexpected dependencies introduced through environment drift or development shortcuts.
2.4	**Utilize Automated Software Inventory Tools**	Utilize software inventory tools, when possible, throughout the enterprise to automate the discovery and documentation of installed software.		●	●	Use automated scanning, registration workflows, or deployment pipelines to detect new agents, tool integrations, state stores, or configuration changes. Reconcile discovered assets with the established inventory.
2.5	**Allowlist Authorized Software**	Use technical controls, such as application allowlisting, to ensure that only authorized software can execute or be accessed. Reassess bi-annually, or more frequently.		●	●	Authorize and approve all agent stack components, including frameworks, memory, tools, and LLMs/SLMs, ensuring that dependencies reflect authorized, monitored services. Enforce controlled deployment via pipelines or container images to prevent ad-hoc library or plug-in installations on runtimes or endpoint devices. Promptly remove deprecated or unapproved software from runtimes to ensure only validated, allowlisted components exist within the agent's execution boundary.
2.6	**Allowlist Authorized Libraries**	Use technical controls to ensure that only authorized software libraries, such as specific .dll, .ocx, and .so files, are allowed to load into a system process. Block unauthorized libraries from loading into a system process. Reassess bi-annually, or more frequently.		●	●	Enforce allowlists for agent-related libraries, including specific versions of orchestration frameworks (e.g., LangChain), tool adapters, MCP clients and servers, and vector store clients. Agents often pull dependencies dynamically; unauthorized libraries can introduce supply-chain vulnerabilities or malicious tool capabilities that compromise the agent's autonomy and security.
2.7	**Allowlist Authorized Scripts**	Use technical controls, such as digital signatures and version control, to ensure that only authorized scripts, such as specific .ps1 and .py files, are allowed to execute. Block unauthorized scripts from executing. Reassess bi-annually, or more frequently.			●	Restrict execution of scripts by agents (e.g., Python code generated by the agent) to only authorized, signed, or sandboxed scripts. Agents capable of generating and executing code (e.g., code interpreters) present a massive risk; allowlisting ensures that they cannot execute arbitrary malicious scripts on the host system.

Model Hosting and Deployment Considerations

Agent software inventories must reflect where agent software is deployed (Deployment Environment) and who controls the runtime that loads or executes that software (Runtime Ownership and Control).

Deployment Environments

- **Cloud-Hosted Agents:** Inventory should include cloud-managed runtimes, serverless functions, container images, package dependencies, and tool adapters used in cloud environments. Integrate with cloud-native inventory and configuration management services to detect drift and unauthorized changes.
- **On-Premises or Private Infrastructure Agents:** Enterprises must track software installed on self-managed servers, orchestration platforms, or local application stacks. Inventory should account for software installed through internal package repositories, custom container images, or manually deployed components.
- **Endpoint or Edge Agents:** Agents embedded in local applications or devices may include local executors, SDKs, plug-ins, or lightweight agent frameworks. Ensure software inventories extend to endpoints using endpoint management tools, MDM systems, or local agent instrumentation.

Runtime Ownership and Control

- **Provider-Managed Runtimes:** Inventory must identify which software components are operated by the provider and which are configurable by the enterprise. Track provider-managed frameworks, tool adapters, and runtime extensions where disclosed, and document organizational configuration components that influence their behavior.
- **Enterprise-Managed Runtimes:** Inventory must include the full software stack deployed in enterprise-operated environments, including custom components, third-party libraries, retrieval modules, and tool clients. Integrate with Continuous Integration/Continuous Delivery (CI/CD) systems to capture software used in deployment pipelines.
- **Local or Embedded Runtimes:** Software used in local runtimes, such as desktop-integrated agents, Integrated Development Environment (IDE) assistants, or mobile-embedded agents, must be inventoried through endpoint discovery or configuration management. Inventory should capture plug-ins, local runtimes, and any libraries enabling local tool execution.

Additional AI Agent Considerations

- Agent frameworks evolve rapidly; inventory should support frequent updates and ensure that deprecated components are removed promptly.
- Tools exposed through the Model Context Protocol (MCP) must be inventoried alongside their software backends and dependencies.
- Multi-agent systems may load additional software dynamically; inventory should capture these dependencies where feasible.

Control 3: Data Protection

Develop processes and technical controls to identify, classify, securely handle, retain, and dispose of data.

AI Agent Applicability

AI agents often process sensitive, confidential, or regulated data as part of planning, tool invocation, retrieval workflows, intermediate reasoning, and multi-step task execution. Unlike stand-alone models, agents maintain working memory, generate intermediate state, and exchange data with external tools and systems. This increases the potential for inadvertent disclosure, long-term retention, cross-context contamination, and improper use of sensitive data.

Agent Data Flows and Memory Architecture

Agents introduce data flows beyond typical application patterns, including:

- **Working memory and scratchpads** used during step-by-step reasoning
- **Long-term memory** storing summaries, retrieved content, or structured state across sessions
- **Vector databases and embedding pipelines** used for retrieval-based reasoning
- **Tool inputs and outputs**, including logs, execution traces, or structured API responses
- **Agent-to-agent messages** in multi-agent workflows
- **Session state and intermediate artifacts** used across multi-step or autonomous actions

Controls must apply to all of these surfaces to ensure classification, isolation, access control, and appropriate retention.

Safeguards

Control 3: Data Protection

Safeguard	Title	Description	IG1	IG2	IG3	AI Agent Applicability
3.1	**Establish and Maintain a Data Management Process**	Establish and maintain a documented data management process. In the process, address data sensitivity, data owner, handling of data, data retention limits, and disposal requirements, based on sensitivity and retention standards for the enterprise. Review and update documentation annually, or when significant enterprise changes occur that could impact this Safeguard.	●	●	●	A robust management process must treat tool outputs with the same rigor as production databases, applying strict access and retention controls to prevent the leakage of the agent's confidential "thought process" and environmental access details. Enforce strict validation on external data ingestion and protect all sensitive tool execution outputs. Agents function as autonomous data pipelines that exist in a continuous loop of consumption and production. On the ingestion side, data retrieved from external APIs or RAG stores must be validated to prevent data poisoning or the ingestion of adversarial content that corrupts the agent's reasoning. Conversely, the artifacts generated by the agent's actions, such as code execution logs, browser snapshots, and intermediate tool responses, often contain highly sensitive ephemeral data or credentials.
3.2	**Establish and Maintain a Data Inventory**	Establish and maintain a data inventory based on the enterprise's data management process. Inventory sensitive data, at a minimum. Review and update inventory annually, at a minimum, with a priority on sensitive data..	●	●	●	Maintain an inventory of agent memory systems, including working memory stores, long-term memory, vector databases, and caching layers. Track data locations, sensitivity, retention behavior, and associated infrastructure.

Control 3: Data Protection

Safeguard	Title	Description	IG1	IG2	IG3	AI Agent Applicability
3.3	**Configure Data Access Control Lists**	Configure data access control lists based on a user's need to know. Apply data access control lists, also known as access permissions, to local and remote file systems, databases, and applications.	●	●	●	Enforce strict least-privilege boundaries for agent actions and apply granular Access Control Lists (ACLs) to all memory stores. Agents effectively act as digital employees; their access to tools, APIs, and data sources must be explicitly bounded by policy to prevent any form of unrestricted or overly broad authority. Simultaneously, the data the agent generates, its working memory, vector indexes, and interaction history, accumulates highly sensitive context. These memory repositories must be protected by rigorous ACLs and retention limits to prevent this sensitive "thought process" data from persisting indefinitely or being accessed by unauthorized entities, such as other agents or lower-privileged users.
3.4	**Enforce Data Retention**	Retain data according to the enterprise's documented data management process. Data retention must include both minimum and maximum timelines.	●	●	●	Enforce strict retention and purging for agent memory, logs, and vector stores, while mandating data minimization for tool inputs and redaction for agent outputs. These components accumulate sensitive "thought patterns" and execution traces; without "need-to-know" filters and auto-deletion, agents risk leaking PII or secrets to third-party APIs or unauthorized users. Stripping requests to the bare minimum and limiting the lifespan of stored context ensures that sensitive information is purged or redacted when no longer necessary, reducing the enterprise's liability and the blast radius of a potential storage or transmission compromise.
3.5	**Securely Dispose of Data**	Securely dispose of data as outlined in the enterprise's documented data management process. Ensure the disposal process and method are commensurate with the data sensitivity.	●	●	●	Implement secure disposal processes for agent memory stores, vector databases, and cached retrieval content when no longer needed or when an agent is decommissioned. Residual data in vector stores or long-term memory can retain sensitive information indefinitely. Secure disposal prevents data recovery and leakage from retired agent systems.
3.6	**Encrypt Data on End-User Devices**	Encrypt data on end-user devices containing sensitive data. Example implementations can include: Windows BitLocker®, Apple FileVault®, Linux® dm-crypt.	●	●	●	Enforce encryption for local agent memory, logs, and tool caches stored on end-user devices running embedded agents. Local agents may cache sensitive retrieval content or conversation history. Encryption protects this data from physical theft or unauthorized access on the device.
3.7	**Establish and Maintain a Data Classification Scheme**	Establish and maintain an overall data classification scheme for the enterprise. Enterprises may use labels, such as "Sensitive," "Confidential," and "Public," and classify their data according to those labels. Review and update the classification scheme annually, or when significant enterprise changes occur that could impact this Safeguard.		●	●	Classify the data types an agent may ingest, store, retrieve, or generate, including tool responses, memory entries, session artifacts, and intermediate reasoning. Classification enables appropriate protection across agent components.
3.8	**Document Data Flows**	Document data flows. Data flow documentation includes service provider data flows and should be based on the enterprise's data management process. Review and update documentation annually, or when significant enterprise changes occur that could impact this Safeguard.		●	●	Document all retrieval sources, vector stores, indexing pipelines, document loaders, and embedding services used by agents. Track where retrieval content originates and which agents may access it.
3.9	**Encrypt Data on Removable Media**	Encrypt data on removable media.		●	●	No Additional AI Agent Guidance
3.10	**Encrypt Sensitive Data at Rest**	Encrypt sensitive data in transit. Example implementations can include: Transport Layer Security (TLS) and Open Secure Shell (OpenSSH).		●	●	Encrypt all agent tool communication channels and agent-to-agent messaging in transit using approved cryptographic standards.

Control 3: Data Protection

Safeguard	Title	Description	IG1	IG2	IG3	AI Agent Applicability
3.11	**Encrypt Sensitive Data at Rest**	Encrypt sensitive data at rest on servers, applications, and databases. Storage-layer encryption, also known as server-side encryption, meets the minimum requirement of this Safeguard. Additional encryption methods may include application-layer encryption, also known as client-side encryption, where access to the data storage device(s) does not permit access to the plain-text data.		●	●	Encrypt all agent memory and enforce secure, isolated storage for credentials and sensitive configurations. Agent architectures store highly sensitive "thought processes" in vector databases and session logs, which must be encrypted to prevent context leakage. Treat API keys, tokens, and other credentials as sensitive data that must never exist in unencrypted form within prompts, code, or logs where they can be extracted by the model. To prevent exposure, utilize secure secret management systems to inject credentials only at runtime, ensuring that plaintext secrets are strictly excluded from the agent's memory and execution traces. This minimizes the presence of sensitive secrets within the agent's data footprint, including source code and configuration files, to maintain the integrity of all data at rest.
3.12	**Segment Data Processing and Storage Based on Sensitivity**	Segment data processing and storage based on the sensitivity of the data. Do not process sensitive data on enterprise assets intended for lower sensitivity data.		●	●	Separate sensitive data across agent workspaces and contexts. Ensure that agents serving different departments, workflows, or trust zones cannot access or leak data across boundaries. Segregate memory stores, retrieval scopes, tool permissions, and execution environments.
3.13	**Deploy a Data Loss Prevention Solution**	Implement an automated tool, such as a host-based Data Loss Prevention (DLP) tool to identify all sensitive data stored, processed, or transmitted through enterprise assets, including those located onsite or at a remote service provider, and update the enterprise's data inventory.			●	Implement monitoring to detect credentials appearing in prompts, memory, logs, RAG content, tool outputs, or model responses in order to promptly detect and respond to an incident. Rotate exposed credentials immediately and remediate the root cause.
3.14	**Log Sensitive Data Access**	Log sensitive data access, including modification and disposal.			●	Log all instances where an agent accesses sensitive data stores, including vector databases, internal APIs, and regulated file systems. Agents can access data at high speed and scale; logging sensitive access provides the necessary audit trail to detect unauthorized data harvesting or policy violations by autonomous agents.

Model Hosting and Deployment Considerations

Agent data protections depend on where the agent executes (Deployment Environment) and who controls the agent runtime (Runtime Ownership and Control). Each combination affects isolation boundaries, trust surfaces, and the location where controls must be enforced.

Deployment Environments

- **Cloud-Hosted Agents:** Ensure that data flows between the agent runtime, memory stores, and tools comply with cloud-specific identity, encryption, and network requirements. Apply protections for multi-tenant boundaries and cloud data governance defined in *CIS Controls v8.1* and the *CIS Controls Cloud Companion Guide*.
- **On-Premises or Private Infrastructure Agents:** Enterprises must secure all storage, compute, and network paths used by agent memory systems, tool connectors, and retrieval pipelines. Physical protections, internal segmentation, and storage controls are fully the enterprise's responsibility.
- **Endpoint or Edge Agents:** Sensitive data may be stored or processed locally on devices. Apply device hardening, endpoint encryption, local DLP, and controlled local storage to prevent unauthorized access, exfiltration, or persistence of sensitive content.

Runtime Ownership and Control

- **Provider-Managed Runtimes:** Data protection controls depend on configuration, policy enforcement, and contractual guarantees. Enterprises must ensure the provider supports data minimization, retention controls, and memory isolation. Tool and retrieval interactions must be constrained through configuration or service-level controls.
- **Enterprise-Managed Runtimes:** Enterprises must implement encryption, identity, access control, segmentation, and retention policies on the agent runtime and all connected stores. Auditability, logging, and DLP protections must be applied directly to the runtime and supporting infrastructure.
- **Local or Embedded Runtimes:** Local runtimes must avoid storing sensitive content unprotected on devices. Encryption, sandboxing, and endpoint security controls must be applied to working memory, local logs, and intermediate tool outputs. Where cloud APIs are used for inference or tool access, ensure outbound data handling complies with enterprise data policies.

Additional AI Agent Considerations

- Multi-agent systems may propagate sensitive data across agent boundaries; classification and routing controls must apply to agent-to-agent messaging.
- Agents with autonomous or long-running workflows may accumulate sensitive information in their working state (“scratchpad”); retention and minimization rules must reflect this behavior.
- Tool responses, especially those involving code execution, browsers, or file operations, can expose data directly or indirectly; ensure they inherit appropriate data protections.

Control 4: Secure Configuration of Enterprise Assets and Software

Establish and maintain the secure configuration of enterprise assets (end-user devices, including portable and mobile; network devices; non-computing/IoT devices; and servers) and software (operating systems and applications).

AI Agent Applicability

AI agents are unique in that their behavior is often defined as much by configuration as by code. Elements that would be hard-coded logic in traditional software, such as decision boundaries, personality guidelines, and tool authorization scopes, are frequently defined in system prompts, configuration files, or model parameters (e.g., temperature, top-p).

Insecure configurations in agents lead to direct security failures. A misconfigured system prompt can allow "jailbreaks" that bypass safety filters. A permissive sandbox configuration for a code-execution tool can allow an agent to break out of its container and compromise the host. Furthermore, because agents are often deployed as ephemeral workloads (e.g., serverless functions or containers), their security relies entirely on the hardened configuration of the base image and the runtime environment.

Therefore, secure configuration for agents must extend beyond the operating system (OS) to include the Prompt-as-Code, the Tool Definition Schemas, and the Runtime Sandbox Policies.

Agent Configuration Surfaces

Agent-specific configuration management must address:

- **System Prompts and Metaprompts** – The foundational instructions that define agent identity, constraints, and objectives. These must be version-controlled and treated as immutable configuration artifacts.
- **Tool Definitions (Schemas)** – The JavaScript Object Notation (JSON)/YAML Ain't Markup Language (YAML) definitions that tell the model how to call tools. Misconfiguration here (e.g., permissive parameter typing) can lead to injection attacks.
- **Inference Parameters** – Settings such as "temperature" (randomness) or "max_tokens" that allow trade-offs between creativity and deterministic stability.
- **Execution Sandboxes** – Configuration of the isolation environments (e.g., Docker, WebAssembly, Firecracker microVMs) where agents execute generated code or browser actions.
- **Orchestration Logic** – Routing rules, step limits (recursion caps), and memory retention settings defined in the agent framework (e.g., LangChain, AutoGen).

Safeguards

Control 4: Secure Configuration of Enterprise Assets and Software

Safeguard	Title	Description	IG1	IG2	IG3	AI Agent Applicability
4.1	**Establish and Maintain a Secure Configuration Process**	Establish and maintain a documented secure configuration process for enterprise assets (end-user devices, including portable and mobile, non-computing/IoT devices, and servers) and software (operating systems and applications). Review and update documentation annually, or when significant enterprise changes occur that could impact this Safeguard.	●	●	●	Maintain an inventory of agent configurations including prompts, tool lists, operational parameters, and policy settings. Version configurations and ensure they are traceable to specific agents and deployments.
4.2	**Establish and Maintain a Secure Configuration Process for Network Infrastructure**	Establish and maintain a documented secure configuration process for network devices. Review and update documentation annually, or when significant enterprise changes occur that could impact this Safeguard.	●	●	●	Enforce hardened, least-privilege network configurations and strict microsegmentation for all agent runtimes. Agents function as high-risk execution nodes that connect disparate systems, effectively bridging internal data with external tools. Their underlying network infrastructure, whether on-premises Virtual Local Area Networks (VLANs) or Virtual Private Clouds (VPCs), must be hardened to prevent them from becoming pivot points for attackers. This requires applying strict microsegmentation to isolate agent workloads, enforcing private subnet usage to deny direct internet exposure, and maintaining rigorous control over security groups. By treating the network configuration as a primary security boundary, enterprises limit the agent's ability to traverse the network or exfiltrate data if compromised.
4.3	**Configure Automatic Session Locking on Enterprise Assets**	Configure automatic session locking on enterprise assets after a defined period of inactivity. For general purpose operating systems, the period must not exceed 15 minutes. For mobile end-user devices, the period must not exceed 2 minutes.	●	●	●	No Additional AI Agent Guidance
4.4	**Implement and Manage a Firewall on Servers**	Implement and manage a firewall on servers, where supported. Example implementations include a virtual firewall, operating system firewall, or a third-party firewall agent.	●	●	●	Configure host-based firewalls on agent orchestration servers to restrict inbound access to authorized users and outbound access to approved LLM APIs and tools. Firewalls prevent unauthorized command-and-control connections and limit the agent's ability to communicate with malicious external services or unapproved internal systems.
4.5	**Implement and Manage a Firewall on End-User Devices**	Implement and manage a host-based firewall or port-filtering tool on end-user devices, with a default-deny rule that drops all traffic except those services and ports that are explicitly allowed.	●	●	●	Use host-based firewalls on endpoints to block unauthorized inbound connections to local agent runtimes and restrict outbound traffic to trusted model endpoints. Local agents often open ports for API access; firewalls prevent these local services from being exposed to the network and thus restrict the agent from exfiltrating data.
4.6	**Securely Manage Enterprise Assets and Software**	Securely manage enterprise assets and software. Example implementations include managing configuration through version-controlled Infrastructure-as-Code (IaC) and accessing administrative interfaces over secure network protocols, such as Secure Shell (SSH) and Hypertext Transfer Protocol Secure (HTTPS). Do not use insecure management protocols, such as Telnet (Teletype Network) and HTTP, unless operationally essential.	●	●	●	Manage agent runtimes and configurations (e.g., prompts, tool definitions) via version-controlled Infrastructure-as-Code (IaC) to ensure they are tamper-evident and reproducible. Isolate code execution tools in ephemeral, hardened sandboxes (such as containers or VMs) with disabled networking and restricted system calls. Treating these tools as untrusted software ensures model-generated code remains contained within a disposable environment, preventing unauthorized host access or lateral movement.
4.7	**Manage Default Accounts on Enterprise Assets and Software**	Manage default accounts on enterprise assets and software, such as root, administrator, and other pre-configured vendor accounts. Example implementations can include: disabling default accounts or making them unusable.	●	●	●	Disable or secure default accounts provided with agent platforms, vector databases, and orchestration tools. Default credentials on agent infrastructure (e.g., a vector database admin console) are a primary vector for attackers to gain control over the agent's memory and logic.

Control 4: Secure Configuration of Enterprise Assets and Software

Safeguard	Title	Description	IG1	IG2	IG3	AI Agent Applicability
4.8	**Uninstall or Disable Unnecessary Services on Enterprise Assets and Software**	Uninstall or disable unnecessary services on enterprise assets and software, such as an unused file sharing service, web application module, or service function.		●	●	Restrict browser automation tools to approved domains and disable unnecessary plugins or JavaScript capabilities. Agents utilizing "headless" browsers for research or navigation present a massive attack surface. These tools should not have unrestricted access to the open web. To prevent drive-by downloads, cross-site scripting (XSS) exploitation, or data exfiltration, administrators must disable unnecessary features, like file uploads or experimental APIs, and enforce "Safe Browsing" policies that allowlist only essential domains. This reduces the risk of an external site hijacking the agent's session. Note: These configurations specifically target the browser tools invoked by the agent. While the agent's decision to browse is in scope, the fundamental hardening of the browser engine should follow established web security standards.
4.9	**Configure Trusted DNS Servers on Enterprise Assets**	Configure trusted DNS servers on network infrastructure. Example implementations include configuring network devices to use enterprise-controlled DNS servers and/or reputable externally accessible DNS servers.		●	●	No Additional AI Agent Guidance
4.10	**Enforce Automatic Device Lockout on Portable End-User Devices**	Enforce automatic device lockout following a predetermined threshold of local failed authentication attempts on portable end-user devices, where supported. For laptops, do not allow more than 20 failed authentication attempts; for tablets and smartphones, no more than 10 failed authentication attempts. Example implementations include Microsoft® InTune Device Lock and Apple® Configuration Profile maxFailedAttempts.		●	●	No Additional AI Agent Guidance
4.11	**Enforce Remote Wipe Capability on Portable End-User Devices**	Remotely wipe enterprise data from enterprise-owned portable end-user devices when deemed appropriate such as lost or stolen devices, or when an individual no longer supports the enterprise.		●	●	No Additional AI Agent Guidance
4.12	**Separate Enterprise Workspaces on Mobile End-User Devices**	Ensure separate enterprise workspaces are used on mobile end-user devices, where supported. Example implementations include using an Apple® Configuration Profile or Android™ Work Profile to separate enterprise applications and data from personal applications and data.			●	No Additional AI Agent Guidance

Model Hosting and Deployment Considerations

Deployment Environments

- **Cloud-Hosted Agents:** Utilize Infrastructure-as-Code (IaC) to define agent workloads, ensuring that serverless functions and containers launch with immutable, hardened configurations. Apply strict security contexts to containers to prevent privilege escalation.
- **On-Premises or Private Infrastructure Agents:** Hardening must extend to the orchestration servers and the local vector databases. Ensure that the "Agent Control Plane" is isolated and that configuration files (prompts/rules) are stored in secure, access-controlled repositories.
- **Endpoint or Edge Agents:** For agents running locally, configurations (such as local tool permissions) must be protected from user tampering. Use OS-level configuration profiles (e.g., MDM) to enforce constraints on local agent runtimes.

Runtime Ownership and Control

- **Provider-Managed Runtimes:** Review the provider's default settings for "Safety" and "Tool Sandbox" isolation. Often, defaults prioritize functionality over security. Enterprises must explicitly configure available toolsets and stricter safety thresholds where allowed.
- **Enterprise-Managed Runtimes:** The enterprise is responsible for the full configuration stack. This includes hardening the base OS, securing the Python/Node.js interpreter, and managing the life cycle of system prompts via a secure deployment pipeline.
- **Local or Embedded Runtimes:** Ensure that embedded agents ship with "locked" configurations that prevent end-users from modifying safety system prompts or enabling high-risk tools (e.g., arbitrary file execution) without administrative override.

Additional AI Agent Considerations

- **Prompt-as-Configuration:** Treat system prompts and agent definitions as software. They should be stored in version control (e.g., Git), subject to peer review, and deployed via automated pipelines, never manually edited in a production console.
- **Configuration Drift:** Agent behavior can drift if the underlying model changes, even if the configuration stays the same. "Configuration Management" for agents implies continuous monitoring of behavioral outputs against the expected baseline.
- **Sandbox Hardening:** Tools that execute code (e.g., Python interpreters) are the highest risk surface. Configuration must explicitly disable networking and restrict filesystem access within these sandboxes unless strictly necessary.

Control 5: Account Management

Use processes and tools to assign and manage authorization to credentials for user accounts, including administrator accounts, as well as service accounts, to enterprise assets and software.

AI Agent Applicability

AI agents interact with systems, services, tools, and data via identities, typically expressed as user accounts, service accounts, API keys, access tokens, or delegated credentials. Agents may:

- Operate under a dedicated service identity
- Inherit identity from a signed-in user
- Use temporary credentials provisioned at runtime
- Manage or request identities on behalf of users
- Interact with tools that have their own identities
- Persist or mishandle identities within memory or logs
- Use a combination of the above when spawning sub-agents, “swarms,” or other multi-agent approaches

Since agents make decisions and take actions across many systems, improper account management can lead to unintended access, privilege escalation, or unauthorized changes to data or configurations.

Agent Account Surfaces

Account management for agents must consider:

- **Agent execution identity** – the primary identity under which the agent runs
- **Tool and API identities** – credentials used when calling external systems or MCP tools
- **User delegation** – user accounts or roles temporarily used by agents
- **Session-linked accounts** – identities bound to specific user sessions or conversations
- **Internal agent sub-process identities** – accounts or tokens used for worker processes, task executors, or retrieval pipelines
- **Credential handling** – how the agent accesses, stores, or forwards credentials in memory, logs, or tool calls

These surfaces must be controlled consistently to prevent misuse.

Safeguards

Control 5: Account Management

Safeguard	Title	Description	IG1	IG2	IG3	AI Agent Applicability
5.1	**Establish and Maintain an Inventory of Accounts**	Establish and maintain an inventory of all accounts managed in the enterprise. The inventory must at a minimum include user, administrator, and service accounts. The inventory, at a minimum, should contain the person's name, username, start/stop dates, and department. Validate that all active accounts are authorized, on a recurring schedule at a minimum quarterly, or more frequently.	●	●	●	Inventory all user accounts, service accounts, API keys, or delegated credentials used by the agent, including those attached to tools or retrieval systems.
5.2	**Use Unique Passwords**	Use unique passwords for all enterprise assets. Best practice implementation includes, at a minimum, an 8-character password for accounts using Multi-Factor Authentication (MFA) and a 14-character password for accounts not using MFA.	●	●	●	Agents integrate distinct architectural components, vector databases (memory), LLM APIs (brain), and execution sandboxes (hands), that must not share credentials. Enforce unique, cryptographically strong authentication tokens for each layer. Specifically, ensure that credentials for vector stores, which hold long-term knowledge, are distinct from those used by runtime engines or tool plugins. This segmentation ensures that if an agent's execution environment is compromised via a tool, the attacker cannot use those same credentials to pivot and exfiltrate the agent's entire history or knowledge base.
5.3	**Disable Dormant Accounts**	Delete or disable any dormant accounts after a period of 45 days of inactivity, where supported.	●	●	●	No Additional AI Agent Guidance
5.4	**Restrict Administrator Privileges to Dedicated Administrator Accounts**	Restrict administrator privileges to dedicated administrator accounts on enterprise assets. Conduct general computing activities, such as internet browsing, email, and productivity suite use, from the user's primary, non-privileged account.	●	●	●	Restrict administrative access to agent orchestration platforms and memory stores to dedicated admin accounts, distinct from standard user or agent identities. Prevents a compromised agent or user account from having full control over the agent ecosystem, limiting the potential damage of a privilege escalation attack.
5.5	**Establish and Maintain an Inventory of Service Accounts**	Establish and maintain an inventory of service accounts. The inventory, at a minimum, must contain department owner, review date, and purpose. Perform service account reviews to validate that all active accounts are authorized, on a recurring schedule at a minimum quarterly, or more frequently.		●	●	Manage tool and API credentials used by agents. Ensure any credentials used to invoke tools, APIs, or MCP capabilities are managed securely. Bind credentials to specific capabilities, rotate them regularly, and store them using secure mechanisms.
5.6	**Centralize Account Management**	Centralize account management through a directory or identity service.		●	●	Manage all agent and tool identities through a centralized identity provider (IdP), prioritizing federated identity and passwordless authentication, such as Certificate Authorities or OAuth with refresh tokens, to mitigate the risks of exploiting passwords or other credentials, including API keys, tokens, and secrets. Centralization provides a single point of control for automated provisioning and simplified revocation, which reduces the risk of orphaned accounts or leaked secrets retaining access to AI assets. Where passwordless methods cannot be implemented, all credentials, including passwords, keys, and tokens, must be rotated on a defined schedule to maintain account security. In multi-agentic architectures, the system must propagate the invoking user's identity and privilege context across the entire agent chain. This propagation prevents unintentional privilege escalation that occurs when interconnected agents utilize service accounts with higher permissions than the end user. Each agent must be restricted to the user's specific authorization, performing nothing more or less than the user is permitted to do. For non-repudiation, every interaction must be identifiable as either a human action or a non-human (agent) action performed on behalf of a user, ensuring that security personnel can distinguish between human intent and autonomous execution traces within the centralized audit log.

Model Hosting and Deployment Considerations

Account management requirements depend on where the agent executes (Deployment Environment) and who controls the agent runtime (Runtime Ownership and Control). Each condition affects how identities are provisioned, stored, and enforced.

Deployment Environments

- **Cloud-Hosted Agents:** Use cloud IAM and identity providers to manage service accounts, roles, and policies. Apply conditional access rules, MFA for user delegation, short-lived credentials, and identity-scoped secrets. Track cloud-managed identities connected to serverless runtimes, managed containers, and hosted execution environments.
- **On-Premises or Private Infrastructure Agents:** Apply enterprise IAM systems, directory services, or local identity providers to manage agent and tool accounts. Ensure service accounts used for agent tasks are managed within enterprise identity governance and tied to monitoring and deprovisioning workflows.
- **Endpoint or Edge Agents:** Use OS-level identities, application sandboxes, device trust, and endpoint management tools to govern agent identities. Restrict local credential access, protect secrets from local exfiltration, and prevent agents from impersonating users without explicit approval.

Runtime Ownership and Control

- **Provider-Managed Runtimes:** Account management depends heavily on provider identity mechanisms. Document how identities are provisioned, limit delegated access, and ensure provider-managed runtimes support credential rotation, scope restrictions, and session controls.
- **Enterprise-Managed Runtimes:** Enterprises must manage all identities used by agent orchestrators, tools, memory systems, and retrieval components. Integrate identity governance platforms and apply automated provisioning, rotation, and monitoring.
- **Local or Embedded Runtimes:** Local agents must not store or cache high-risk credentials on devices. Manage local execution identity within the OS and restrict outbound service access unless proper credentials and network controls are applied.

Additional AI Agent Considerations

- Agents must never autonomously create new user or service accounts unless specifically authorized with controlled delegation.
- Delegated user identities must include time limits, scope limits, and revocation paths.
- Tool identities may require different life cycles than agent identities; inventory and manage them separately.
- Avoid embedding credentials in prompts, memory, RAG sources, or intermediate reasoning.

Control 6: Access Control Management

Use processes and tools to create, assign, manage, and revoke access credentials and privileges for user, administrator, and service accounts for enterprise assets and software.

AI Agent Applicability

AI agents act on behalf of users and services, creating new access control surfaces that extend beyond traditional software boundaries. Agents may hold broad privileges, inherit access from delegated user identities, or operate under dedicated service accounts.

Since agents make autonomous decisions, weak access boundaries can lead to privilege escalation, unauthorized data access, or tool misuse. Effective access control for agents requires managing both external access surfaces (e.g., access to data, APIs, and tools) and internal decision surfaces (e.g., how the agent uses those capabilities).

While agents technically operate as “Service Accounts” (aka Non-Human Identities or NHIs, or Machine Identities), enterprises must avoid legacy practices like assigning static, over-privileged keys to them. Unlike a predictable script, an autonomous agent is susceptible to prompt injection. Therefore, while the agent identity (the entity) may be persistent, its agent credentials (the keys) must be ephemeral and strictly scoped to the immediate task.

Intra-Agent Access Control

Beyond standard login permissions, agents introduce these unique internal access control challenges:

- **Trust boundaries** – what the agent itself is trusted to do versus what it must delegate to a human
- **Tool authorization** – which specific tools (e.g., “Write to Database” vs. “Read from Database”) the agent may invoke
- **Memory access** – what historical context or vector store segments the agent can read or write
- **Delegation** – whether agent actions require “human-in-the-loop” approval or step-up authentication

Agent Access Control Surfaces

AI agents introduce several key access surfaces that must be governed:

- **Tool access and entitlements** – which tools an agent can call, under what constraints, and with what identity
- **Data access boundaries** – which memory stores, vector databases, or retrieval sources the agent may read or write
- **User-to-agent interactions** – how user identity, role, and scope influence the agent’s behavior (e.g., preventing a “basic user” from asking an agent to perform “admin” tasks)
- **Agent-to-service interactions** – service accounts, API keys, and backend integrations used when the agent acts on external systems
- **Action execution controls** – whether specific actions require user approval, guardrails, or policy checks
- **Autonomous task boundaries** – what tasks the agent may initiate, repeat, or escalate without human involvement

Safeguards

Control 6: Access Control Management

Safeguard	Title	Description	IG1	IG2	IG3	AI Agent Applicability
6.1	**Establish an Access Granting Process**	Establish and follow a documented process, preferably automated, for granting access to enterprise assets upon new hire or role change of a user.	●	●	●	Provision persistent, unique identities for agents, using ephemeral credentials where possible or managed secrets where necessary. Agents must operate as distinct digital entities with a persistent identity (e.g., a unique Client ID) to ensure accountability and traceability. While the identity persists, the access methods should ideally be ephemeral (short-lived tokens) to eliminate credential theft risks. Where infrastructure limitations require static credentials (e.g., legacy API keys), these must be strictly governed via secure secret management systems with automated rotation policies. This ensures that every agent action is attributable to a specific identity, regardless of the underlying authentication method.
6.2	**Establish an Access Revoking Process**	Establish and follow a process, preferably automated, for revoking access to enterprise assets, through disabling accounts immediately upon termination, rights revocation, or role change of a user. Disabling accounts, instead of deleting accounts, may be necessary to preserve audit trails.	●	●	●	Immediately revoke identities and purge entitlements upon agent retirement, reconfiguration, or scope change. Agents leave behind "digital exhaust" in the form of service accounts and API keys. Without a rigorous offboarding process, these credentials become "orphaned" entitlements that persist long after the agent has stopped running. A formal revocation workflow must ensure that whenever an agent is decommissioned or moved to a new environment, its specific identity is disabled and all associated tokens are invalidated/rotated to prevent "zombie" access by attackers.
6.3	**Require MFA for Externally-Exposed Applications**	Require all externally-exposed enterprise or third-party applications to enforce MFA, where supported. Enforcing MFA through a directory service or SSO provider is a satisfactory implementation of this Safeguard.	●	●	●	Enforce MFA for access to any externally exposed agent interfaces, management consoles, or tool gateways. Agents often have powerful capabilities; MFA adds a critical layer of defense against unauthorized access to these high-risk interfaces, protecting against credential theft.
6.4	**Require MFA for Remote Network Access**	Require MFA for remote network access.	●	●	●	No Additional AI Agent Guidance
6.5	**Require MFA for Administrative Access**	Require MFA for all administrative access accounts, where supported, on all enterprise assets, whether managed on-site or through a service provider.	●	●	●	Enforce MFA for all administrative actions within agent platforms, including policy changes and configuration updates. Protects critical agent governance functions from compromise, ensuring that only authorized and verified administrators can modify agent behavior or security settings.
6.6	**Establish and Maintain an Inventory of Authentication and Authorization Systems**	Establish and maintain an inventory of the enterprise's authentication and authorization systems, including those hosted on-site or at a remote service provider. Review and update the inventory, at a minimum, annually, or more frequently.		●	●	Maintain an inventory of all authentication systems used by agents, including API gateway auth, service meshes, and tool-specific auth mechanisms. Ensures visibility into all access control pathways for agents, preventing "shadow" authentication methods that could be bypassed to gain unauthorized access to agents.
6.7	**Centralize Access Control**	Centralize access control for all enterprise assets through a directory service or SSO provider, where supported.		●	●	Manage agent identities via central directory services or single sign-on (SSO), avoiding scattered local credentials. Agent identities should not exist as "ghost" accounts scattered across local configuration files or proprietary tool databases. Instead, they must be provisioned and managed within the enterprise's central directory service (e.g., Active Directory, Lightweight Directory Access Protocol (LDAP), or Identity Provider (IdP)) just like human employees. This centralization ensures that agents inherit global security policies, such as password complexity, rotation schedules, and immediate suspension capability, allowing security teams to audit and control agent access through a single pane of glass rather than attempting to manage disparate credentials across the ecosystem.

Control 6: Access Control Management

Safeguard	Title	Description	IG1	IG2	IG3	AI Agent Applicability
6.8	Define and Maintain Role-Based Access Control	Define and maintain role-based access control, through determining and documenting the access rights necessary for each role within the enterprise to successfully carry out its assigned duties. Perform access control reviews of enterprise assets to validate that all privileges are authorized, on a recurring schedule at a minimum annually, or more frequently.			●	Assign granular role-based access control (RBAC) roles to agent identities and bind delegated user permissions to ephemeral session contexts. Agent identities must map to defined organizational roles with least-privilege sets, explicitly defining which roles are permitted to invoke specific tools or APIs. When acting on behalf of a human, the framework must enforce Delegated RBAC (where agent authority is cryptographically bound to the user session). High-impact operations, such as modifying production data, executing code, or initiating financial transactions, require Attribute-Based Access Control (ABAC) overlays or Just-in-Time (JIT) role elevation via explicit policy checks or human-in-the-loop approval. In multi-agentic systems, role-defined boundaries must govern peer-to-peer communication and task delegation to prevent low-privilege agents from escalating authority by prompting high-privilege peers. RBAC policies must also enforce data ingress limits, ensuring that agents do not ingest data classified above their role's clearance level, to prevent agents from exceeding their defined clearance, hallucinating unauthorized write operations, or persisting authority beyond the active workflow.

Model Hosting and Deployment Considerations

Access control enforcement varies based on both where the agent executes (Deployment Environment) and who operates the agent runtime (Runtime Ownership and Control).

Deployment Environments

- **Cloud-Hosted Agents:** Apply cloud IAM roles, service identities, and network segmentation to constrain agent actions. Use "Workload Identity" features to issue short-lived tokens to agents rather than static keys. Ensure data stores and tools are segmented by trust level.
- **On-Premises or Private Infrastructure Agents:** Implement access controls using organizational IAM, network segmentation, and internal authorization systems. Ensure local tools and databases have explicit permissions governing agent use.
- **Endpoint or Edge Agents:** Use device identity, local OS controls, and application sandboxing to restrict tool access. Prevent local agents from accessing files or credentials beyond authorized scopes.

Runtime Ownership and Control

- **Provider-Managed Runtimes:** Access decisions often depend on provider configurations. Restrict which tools the provider-hosted agent can call and apply policy boundaries via configuration. Validate provider capabilities for least privilege.
- **Enterprise-Managed Runtimes:** Enterprises must enforce access controls directly through agent code, execution platforms, and internal policies. Apply least privilege to service accounts, retrieval systems, and agent-to-agent messaging.
- **Local or Embedded Runtimes:** Access must rely on local device identity and OS-level controls. Prevent local runtimes from accessing unauthorized files or credentials.

Additional AI Agent Considerations

- **Context-Aware Scoping:** Access control should be context-aware. An agent acting as a “Developer” should only have access to the specific repository relevant to the current ticket, not all repositories.
- **Delegated Authority:** When an agent acts on behalf of a user, it should use **Delegated Authorization** (e.g., On-Behalf-Of flows) rather than a broad service account. This ensures the agent’s permissions are capped by the user’s own entitlements.
- **Credential Exclusion from Checkpoints:** Agents often serialize their state (memory and scratchpads) to disk to “pause” and “resume” workflows. Ensure that active access tokens are stripped from this state before serialization to prevent creating static secrets on disk/database.
- **Attribution and Non-Repudiation:** Access logs must distinguish between a direct user action and an agent action. Use distinct user-agent strings or dedicated identity claims (e.g., “acted_by: agent_id”) to ensure forensic attribution.
- **Tool-Level Credential Isolation:** Avoid sharing a single “Agent Identity” across all tools. Where possible, inject credentials only into the specific tool environment (sandbox) that needs them, ensuring the “Weather Tool” cannot access the credentials of the “Database Tool”.
- **Human-in-the-Loop Gates:** For sensitive operations (e.g., financial transfers, data deletion), access control should require an out-of-band human approval step.

Control 7: Continuous Vulnerability Management

Develop a plan to continuously assess and track vulnerabilities on all enterprise assets within the enterprise's infrastructure, in order to remediate, and minimize, the window of opportunity for attackers. Monitor public and private industry sources for new threat and vulnerability information.

AI Agent Applicability

AI agents introduce unique vulnerability surfaces, including orchestration frameworks, tool integrations, memory systems, retrieval pipelines, embedding services, execution environments, agent-specific SDKs, and model clients. These components evolve quickly and may introduce vulnerabilities through third-party libraries, insecure tool adapters, outdated memory pipelines, or unsafe execution environments.

Agents may also interact with surfaces that dramatically expand the exposure to vulnerabilities and require fast remediation, such as code interpreters, browsers, or file handlers. Since agents often run autonomously or semi-autonomously, unpatched vulnerabilities can enable privilege escalation, data compromise, or remote code execution through agent-driven actions.

Agent Vulnerability Surfaces

Vulnerability management must apply to the following AI agent–specific components:

- **Agent frameworks and orchestration layers**
- **Tooling executors** (e.g., code execution, browser automation, file operations)
- **Third-party tool libraries and SDKs**
- **Vector-store clients, embedding libraries, and retrieval pipelines**
- **Runtime plug-ins, extensions, and custom modules**
- **Model clients, inference SDKs, and transport libraries**
- **Agent control logic and planning components**
- **Sandbox environments for execution tools**
- **Endpoints where agents run** (devices, edge systems, local interpreters)

Each introduces dependencies and third-party code that must be continuously evaluated.

Safeguards

Control 7: Continuous Vulnerability Management

Safeguard	Title	Description	IG1	IG2	IG3	AI Agent Applicability
7.1	**Establish and Maintain a Vulnerability Management Process**	Establish and maintain a documented vulnerability management process for enterprise assets. Review and update documentation annually, or when significant enterprise changes occur that could impact this Safeguard.	●	●	●	Inventory vulnerability sources for agent components. Identify the sources of vulnerabilities affecting agent frameworks, tool adapters, memory systems, SDKs, and runtime environments. Track disclosure channels such as vendor advisories, package-manager feeds, Common Vulnerabilities and Exposures (CVE) databases, or upstream repositories.
7.2	**Establish and Maintain a Remediation Process**	Establish and maintain a risk-based remediation strategy documented in a remediation process, with monthly, or more frequent, reviews.	●	●	●	Conduct risk-based prioritization of agent vulnerabilities. Prioritize remediation based on exploitability, agent privileges, sensitivity of accessed data, and potential operational impact. High-risk tools and execution environments should receive expedited remediation.
7.3	**Perform Automated Operating System Patch Management**	Perform operating system updates on enterprise assets through automated patch management on a monthly, or more frequent, basis.	●	●	●	No Additional AI Agent Guidance
7.4	**Perform Automated Application Patch Management**	Perform application updates on enterprise assets through automated patch management on a monthly, or more frequent, basis.	●	●	●	Enforce automated patching for agent frameworks, SDKs, and tool execution runtimes. Agents rely on a complex software stack comprising logic frameworks (e.g., planners, model clients) and distinct execution environments (e.g., code interpreters, browser engines). Security flaws in the framework can enable logic bypasses or injection attacks, while unpatched execution tools can allow malicious code to escape sandboxes and compromise the underlying host. Therefore, the patch management process must synchronize updates across this entire dependency chain. Automation is critical to close vulnerability windows immediately, but it must be paired with regression testing to ensure that updates do not inadvertently break agent behavior or disable safety guardrails.
7.5	**Perform Automated Vulnerability Scans of Internal Enterprise Assets**	Perform automated vulnerability scans of internal enterprise assets on a quarterly, or more frequent, basis. Conduct both authenticated and unauthenticated scans.		●	●	Continuously scan agent codebases, containers, model clients, and transitive dependencies for known vulnerabilities. Agent architectures rely on deep, complex dependency trees, including heavy frameworks, model SDKs, vector store connectors, and transport clients (gRPC/REST). These components often introduce significant transitive risk, where a vulnerability in a buried dependency exposes the entire agent. Automated vulnerability scanning must extend beyond the core application code to inspect container images, runtime environments, and the full supply chain of third-party libraries. This is critical because vulnerabilities in low-level inference libraries or communication layers can be exploited to hijack agent logic or exfiltrate sensitive memory. Continuous monitoring ensures that as new CVEs are discovered in the rapidly evolving AI ecosystem, the enterprise can identify and remediate risks in the agent's stack before they are targeted.
7.6	**Perform Automated Vulnerability Scans of Externally-Exposed Enterprise Assets**	Perform automated vulnerability scans of externally-exposed enterprise assets. Perform scans on a monthly, or more frequent, basis.		●	●	Regularly scan externally-exposed agent APIs and interfaces for vulnerabilities. Agents exposing public endpoints are prime targets; scanning identifies vulnerabilities in the agent interface or underlying stack before attackers can exploit them.
7.7	**Remediate Detected Vulnerabilities**	Remediate detected vulnerabilities in software through processes and tooling on a monthly, or more frequent, basis, based on the remediation process.		●	●	Remediate vulnerabilities in agent memory and retrieval pipelines. Patch or reconfigure retrieval pipelines, embedding services, indexing jobs, or memory systems affected by vulnerabilities. Validate compatibility before applying upgrades to avoid memory corruption or retrieval errors.

Model Hosting and Deployment Considerations

Vulnerability management requirements depend on both where the agent executes and who operates the runtime.

Deployment Environments

- **Cloud-Hosted Agents:** Use cloud-native vulnerability scanning and patching services to monitor container images, serverless runtimes, and managed compute surfaces. Track vulnerabilities in cloud-managed agent components and ensure remediation timelines align with provider recommendations.
- **On-Premises or Private Infrastructure Agents:** Apply enterprise patching workflows to agent runtimes, libraries, retrieval systems, and local vector stores. Ensure internal vulnerability scanners cover build systems, package repositories, and deployment pipelines.
- **Endpoint or Edge Agents:** Agents embedded on endpoints may rely on local execution engines, libraries, or browser components. Apply endpoint vulnerability scanning, Operating System (OS) patching, local sandbox updates, and application hardening. Ensure that local vulnerabilities do not propagate into remote tool interactions.

Runtime Ownership and Control

- **Provider-Managed Runtimes:** Monitor provider advisories for vulnerabilities affecting agent runtimes. Ensure configuration changes or mitigations are applied where customer-controlled. Validate that provider-managed runtimes support timely patching, dependency updates, and secret rotation.
- **Enterprise-Managed Runtimes:** Enterprises must scan and patch all components of the agent runtime, including frameworks, tools, retrieval engines, embeddings, storage clients, and execution sandboxes. Integrate agent builds into CI/CD scanning pipelines.
- **Local or Embedded Runtimes:** Ensure endpoints receive timely OS and application updates. Apply sandbox hardening and secure local interpreters or tool runtimes. Prevent outdated local components from exposing credentials or data during tool execution.

Additional AI Agent Considerations

- Vulnerabilities in agent tools (especially code execution or browsers) can be exploited through crafted inputs delivered via the agent itself.
- Retrieval pipelines may import vulnerable or malicious content into memory or prompt contexts; monitor integrity and validation.
- Agents that synthesize or execute code require strict monitoring of interpreter vulnerabilities and sandbox protections.

Control 8: Audit Log Management

Collect, alert, review, and retain audit logs of events that could help detect, understand, or recover from an attack.

AI Agent Applicability

AI agents create unique logging surfaces because they operate through reasoning steps, tool calls, retrieval flows, memory interactions, and autonomous or semi-autonomous actions. Agents may generate logs that include prompts, retrieved content, tool outputs, intermediate reasoning, or execution traces, some of which may contain sensitive or regulated information.

Audit logs are essential for:

- Investigating unintended agent actions
- Understanding how decisions were made
- Detecting malicious tool use or unauthorized data access
- Reconstructing multi-step agent workflows
- Reviewing compliance with internal policies

However, logging must be done without exposing sensitive data, model-internal content, or user-provided information.

Agent Logging Surfaces

Agent-specific logs may include:

- **Tool invocation logs** – which tools were invoked, with what parameters, and by which agent
- **Memory interactions** – reads/writes to working memory, long-term memory, vector stores, or caches
- **Retrieval activities** – documents retrieved, embedding lookups, index queries, or metadata
- **Agent action traces** – planning steps, decision branches, policy evaluation, or guardrail outcomes
- **Model interactions** – calls to LLM/SLM endpoints, including inputs/outputs where allowed by policy
- **User-agent interaction logs** – user requests, session IDs, and/or delegation context
- **Execution tool logs** – outputs from code interpreters, browsers, file handlers, or sandboxed environments
- **System and infrastructure logs** – runtime events, errors, resource usage, and/or deployment life cycle

All must be governed by appropriate retention, redaction, security, and access rules.

Safeguards

Control 8: Audit Log Management

Safeguard	Title	Description	IG1	IG2	IG3	AI Agent Applicability
8.1	**Establish and Maintain an Audit Log Management Process**	Establish and maintain a documented audit log management process that defines the enterprise's logging requirements. At a minimum, address the collection, review, and retention of audit logs for enterprise assets. Review and update documentation annually, or when significant enterprise changes occur that could impact this Safeguard.	●	●	●	Establish an audit log management process that logs all agent reasoning, tool interactions, and memory access while enforcing strict redaction and anti-tamper controls. Agents operate as non-deterministic systems; understanding "why" they took an action requires deep visibility into their reasoning chains, tool inputs/outputs, and memory retrieval operations. The management process must mandate this comprehensive traceability for forensics and compliance. However, because agents often process raw PII or credentials in working memory, logs must be vigorously sanitized, redacting secrets and sensitive data before they are written to disk. Furthermore, the process must secure logs against tampering (specifically preventing the agent itself from modifying its own history) and require re-validation of logging pipelines after any configuration update to ensure the "black box" remains auditable.
8.2	**Collect Audit Logs**	Collect audit logs. Ensure that logging, per the enterprise's audit log management process, has been enabled across enterprise assets.	●	●	●	Log agent access decisions by recording agent tool calls, data accesses, authorization decisions, and attempted policy violations.
8.3	**Ensure Adequate Audit Log Storage**	Ensure that logging destinations maintain adequate storage to comply with the enterprise's audit log management process.	●	●	●	No Additional AI Agent Guidance
8.4	**Standardize Time Synchronization**	Standardize time synchronization. Configure at least two synchronized time sources across enterprise assets, where supported..		●	●	No Additional AI Agent Guidance
8.5	**Collect Detailed Audit Logs**	Configure detailed audit logging for enterprise assets containing sensitive data. Include event source, date, username, timestamp, source addresses, destination addresses, and other useful elements that could assist in a forensic investigation.		●	●	Log and preserve comprehensive agent activity, including tool call histories, memory interaction logs, retrieval traces, and model inputs/outputs where policy permits, to enable forensic investigation and behavior reconstruction. Record all tool calls, capturing timestamps, identity used, parameters passed, and outcomes, alongside significant reads and writes to working memory, long-term memory, and vector stores. Retrieval events must be documented by recording the sources queried and the high-level nature of retrieved content without exposing raw sensitive data. These detailed logs allow security teams to reconstruct reasoning chains and executed actions, ensuring a forensic path to detect unauthorized activity or identify the origin of autonomous decisions.
8.6	**Collect DNS Query Audit Logs**	Collect DNS query audit logs on enterprise assets, where appropriate and supported.		●	●	No Additional AI Agent Guidance
8.7	**Collect URL Request Audit Logs**	Collect URL request audit logs on enterprise assets, where appropriate and supported.		●	●	Log agent-driven browsing and URL-fetch activity to detect suspicious patterns, unexpected domains, anomalous downloads, or attempted access to blocked content.
8.8	**Collect Command-Line Audit Logs**	Collect command-line audit logs. Example implementations include collecting audit logs from PowerShell®, BASH™, and remote administrative terminals.		●	●	Log command-line activity within agent execution environments (e.g., code interpreter sandboxes) to track executed scripts and commands. Provides visibility into exactly what code or commands an agent executed, which is critical for identifying malicious or unintended actions taken by the agent.
8.9	**Centralize Audit Logs**	Centralize, to the extent possible, audit log collection and retention across enterprise assets in accordance with the documented audit log management process. Example implementations primarily include leveraging a SIEM tool to centralize multiple log sources.		●	●	Forward agent logs to centralized logging or Security Information and Event Management (SIEM) systems. Ensure logs from agent runtimes, tools, memory systems, and retrieval engines are correlated for end-to-end traceability.

Control 8: Audit Log Management

Safeguard	Title	Description	IG1	IG2	IG3	AI Agent Applicability
8.10	Retain Audit Logs	Retain audit logs across enterprise assets for a minimum of 90 days.		●	●	Set retention periods for agent logs consistent with enterprise, regulatory, or audit requirements. Ensure logs containing sensitive metadata are retained only as long as necessary.
8.11	Conduct Audit Log Reviews	Conduct reviews of audit logs to detect anomalies or abnormal events that could indicate a potential threat. Conduct reviews on a weekly, or more frequent, basis.		●	●	Regularly review logs to detect identity misuse, behavioral anomalies, and unexpected decision paths. Agents operate with significant autonomy, making standard log reviews insufficient. Security teams must analyze logs not just for error rates, but for "behavioral drift." This involves detecting anomalies in identity usage (e.g., unexpected privilege elevation or delegated access misuse) and operational logic (e.g., accessing sensitive memory retrieval paths or invoking tools in unapproved sequences). Unlike deterministic software, agents may "decide" to take high-risk actions that technically succeed but violate intent. Regular reviews are essential to identify these subtle patterns of misuse, hallucination, or prompt injection, allowing enterprises to tune access policies and interrupt potential "confused deputy" attacks before they escalate into full compromises.
8.12	Collect Service Provider Logs	Collect service provider logs, where supported. Example implementations include collecting authentication and authorization events, data creation and disposal events, and user management events.			●	Collect and ingest logs from third-party agent platforms and model providers to maintain a complete audit trail. Provider logs contain essential visibility into the hosted portion of agent execution; without them, the enterprise has blind spots in agent activity and decision-making.

Model Hosting and Deployment Considerations

Logging responsibilities differ depending on where the agent runs and who controls the agent runtime.

Deployment Environments

- **Cloud-Hosted Agents:** Use cloud-native logging facilities to capture agent runtime logs, tool invocation logs, memory system activity, and retrieval events. Ensure logs remain within controlled boundaries and comply with cloud provider storage, encryption, and retention standards.
- **On-Premises or Private Infrastructure Agents:** Integrate agent logs with internal SIEM, syslog, or centralized log aggregation services. Ensure internal runtime components such as retrieval engines, vector stores, and execution sandboxes produce logs compatible with enterprise audit systems.
- **Endpoint or Edge Agents:** Capture logs locally but ensure they are forwarded to central systems when connectivity is available. Protect local logs against unauthorized access and ensure that sensitive data is not stored in clear text on endpoints.

Runtime Ownership and Control

- **Provider-Managed Runtimes:** Understand which logs are generated and accessible from the provider-managed agent environment. Configure provider options for log retention, redaction, and export. Ensure that sufficient metadata is available to support forensic analysis.
- **Enterprise-Managed Runtimes:** Enterprises must implement full logging for agent decisions, tool calls, retrieval, memory usage, and model invocations. Ensure all runtime-managed components emit structured logs and that redaction occurs consistently.
- **Local or Embedded Runtimes:** Local runtimes must log activity without exposing sensitive data to device storage. Use secure device logging mechanisms and ensure logs flow back to enterprise systems where appropriate.

Additional AI Agent Considerations

- Agent logs may include policy decisions or guardrail outcomes; these can reveal essential forensic details without exposing raw model inputs.
- Execution tools (code interpreters, browser automation) represent high-risk surfaces; detailed, redacted logs are necessary for incident investigation.
- Retrieval systems may log sensitive document identifiers; ensure policies prevent logging full document content.

Control 9: Email and Web Browser Protections

Improve protections and detections of threats from email and web vectors, as these are opportunities for attackers to manipulate human behavior through direct engagement.

AI Agent Applicability

While AI agents do not typically use human-facing email or web browsers, many agents interact with browser automation tools, URL fetchers, web-scraping utilities, or email APIs as part of their operational capabilities. These tools introduce risks similar to those faced by human-operated browsers: exposure to malicious content, untrusted downloads, phishing pages, or scripts that could exploit browser engines or parsing libraries.

Agents that process email messages, retrieve URLs, or browse the web autonomously can inadvertently pull harmful content into internal systems or expose sensitive data to external sites. Agents may also mishandle HTML, JavaScript, MIME content, or attachments, potentially triggering vulnerabilities in tool libraries or allowing crafted content to influence agent reasoning.

Agent-Specific Exposure Points

Agent workflows may involve:

- Browser automation tools for data gathering, testing, or navigation
- AI-powered browsers (e.g., Perplexity Comet, ChatGPT Atlas)
- HTTPS/URL-fetch tools retrieving information from arbitrary web resources
- File download mechanisms used by agents during planning or tool invocation
- Email-retrieval APIs or mailbox integrations for processing inbound messages
- HTML and document parsers used in extraction or summarization workflows

These components must be hardened to prevent malicious content from impacting agent behavior or underlying systems. AI tools reading emails or other messages on behalf of users or as part of automated activities are subject to prompt injection and other attacks that can be embedded in the messages.

Safeguards

Control 9: Email and Web Browser Protections

Safeguard	Title	Description	IG1	IG2	IG3	AI Agent Applicability
9.1	**Ensure Use of Only Fully Supported Browsers and Email Clients**	Ensure only fully supported browsers and email clients are allowed to execute in the enterprise, only using the latest version of browsers and email clients provided through the vendor.	●	●	●	Ensure browsers, AI powered browsers, headless browser engines, HTML parsers, PDF readers, and document processors used by agents are up-to-date and hardened against known exploitation techniques.
9.2	**Use DNS Filtering Services**	Use DNS filtering services on all end-user devices, including remote and on-premises assets, to block access to known malicious domains.	●	●	●	Ensure DNS filtering is in place anywhere agents run to block agent runtimes from resolving known malicious or unapproved domains. This prevents agents from inadvertently connecting to malware distribution sites, phishing domains, or unapproved external services during autonomous web browsing.

Control 9: Email and Web Browser Protections

Safeguard	Title	Description	IG1	IG2	IG3	AI Agent Applicability
9.3	Maintain and Enforce Network-Based URL Filters	Enforce and update network-based URL filters to limit an enterprise asset from connecting to potentially malicious or unapproved websites. Example implementations include category-based filtering, reputation-based filtering, or through the use of block lists. Enforce filters for all enterprise assets.		●	●	Enforce strict domain via allowlisting and active content inspection for all agent outbound network traffic. Agents often act as autonomous browsers or API clients, making them susceptible to interacting with malicious infrastructure. Unlike human users who might notice a suspicious URL, agents will blindly execute instructions. Therefore, network policy must rely on strict allowlists rather than blacklists, permitting access only to explicitly authorized APIs and domains required for the workflow. Beyond simple connectivity, employ active content inspection and safe browsing filters to analyze the payload of retrieved data. This prevents agents from inadvertently downloading malware, communicating with command-and-control servers, or exfiltrating data to unapproved endpoints.
9.4	Restrict Unnecessary or Unauthorized Browser and Email Client Extensions	Restrict, either through uninstalling or disabling, any unauthorized or unnecessary browser or email client plugins, extensions, and add-on applications.		●	●	Restrict the use of unauthorized extensions or plug-ins in browsers used by agents for automation. Malicious extensions can intercept agent browsing data or inject unauthorized commands; restricting them reduces the attack surface of the agent's browser environment.
9.5	Implement DMARC	To lower the chance of spoofed or modified emails from valid domains, implement DMARC policy and verification, starting with implementing the Sender Policy Framework (SPF) and the DomainKeys Identified Mail (DKIM) standards.		●	●	While DMARC is not directly applicable to AI Agents, DMARC reduces spoofing used to trick agents into granting malicious access, installing malware, or sending sensitive data to unauthorized destinations and should be considered a protective layer to ensure robust agent safety.
9.6	Block Unnecessary File Types	Block unnecessary file types attempting to enter the enterprise's email gateway.		●	●	Block dangerous file downloads and enforce strict sanitization of active content in retrieved data. Agents act as autonomous retrieval systems, often downloading files or scraping web pages without human oversight. This creates a direct pipeline for malware delivery. Controls must strictly block the download of high-risk file types (e.g., executables, archives, scripts) that are not critical to the business function. Furthermore, for allowed file types, such as HTML or PDF, the system must enforce rigorous content sanitization, stripping out active components like JavaScript, embedded objects, or tracking pixels before the content is processed by the agent. This prevents adversaries from using "active content" attacks to execute code within the agent's runtime or hijack its session.
9.7	Deploy and Maintain Email Server Anti-Malware Protections	Deploy and maintain email server anti-malware protections, such as attachment scanning and/or sandboxing.			●	Apply security controls to email APIs and mailbox access. When agents use email APIs, enforce protections such as attachment filtering, safe-link rewriting, malicious-URL detection, and phishing analysis. Ensure email content is sanitized before being processed or summarized by agents.

Model Hosting and Deployment Considerations

Deployment Environments

- **Cloud-Hosted Agents:** Restrict outbound HTTP requests using cloud network policies, firewall rules, and virtual private cloud (VPC) configurations. Ensure browser automation or URL-fetching tools run within sandboxed environments. Apply cloud-native URL filtering or inspection if available.
- **On-Premises or Private Infrastructure Agents:** Use network security controls such as proxies, URL filtering, and DNS security to restrict agent web access. Apply sandboxing or isolated execution environments for browser automation tools and parsing libraries.
- **Endpoint or Edge Agents:** Ensure that local agents cannot access untrusted sites, open malicious attachments, or interact with local browsers in unsafe ways. Apply OS-level protections, browser hardening, and endpoint security inspection for downloaded content.

Runtime Ownership and Control

- **Provider-Managed Runtimes:** Understand which browsing or URL-fetching capabilities are available in the provider-managed environment. Use configuration and allowlists to restrict agent web activity. Validate provider controls for malware inspection, link safety, and content filtering.
- **Enterprise-Managed Runtimes:** Apply enterprise-wide web protections (e.g., secure web gateways, proxy rules, DNS security) to agent-run browsers or retrieval tools. Ensure tool containers or sandboxes are isolated and regularly updated.
- **Local or Embedded Runtimes:** Local runtimes must not allow agents to launch full browsers or download files without explicit authorization. Apply device-level controls to prevent local exploitation or infection.

Additional AI Agent Considerations

- Agents interacting with external web content should not blindly trust retrieved information, which can influence reasoning or contaminate memory. Employ defense-in-depth validation to scan and sanitize all untrusted content prior to ingestion.
- HTML, PDF, and document parsers used by agents are common vulnerability targets; patch them promptly.
- Email-processing agents must handle phishing, spoofing, and malicious attachments with the same rigor applied to human inboxes.
- Agents should not be permitted to autonomously navigate or explore the public web without strong guardrails.
- Enforce strict trust boundaries by treating all tools that access the web as untrusted components, requiring their outputs to undergo independent verification before they are permitted to influence the agent's state or reasoning.

Control 10: Malware Defenses

Prevent or control the installation, spread, and execution of malicious applications, code, or scripts on enterprise assets.

AI Agent Applicability

AI agents introduce unique malware exposure pathways because they may autonomously retrieve files, generate code, run code through interpreters or execution tools, or interact with untrusted external systems. Agents can also inadvertently propagate malicious content into retrieval pipelines, memory systems, file stores, or downstream users.

Agents operating in browser tools, file-handling tools, or code execution environments must be treated as potential sources of malware ingestion or execution. Moreover, because agents often rely on high-level abstractions and tool APIs, they may be unaware that retrieved content or executed code is harmful, allowing malicious payloads to run without human intervention.

Agent Malware Exposure Points

Malware risks in agent systems may arise through:

- **File retrieval workflows** – downloading documents, images, scripts, archives, or executables
- **Browser automation or URL-fetch tools** – loading malicious web pages that exploit parsing engines
- **Code execution tools** – running or evaluating code generated by the agent or retrieved from external sources
- **File analysis or summarization tools** – processing malicious PDFs, HTML, scripts, or embedded payloads
- **Local runtime exposure** – malware targeting local agent runtimes, device storage, or local credentials
- **RAG ingestion pipelines** – malicious content being added to vector stores or memory systems
- **Cross-agent contamination** – one agent introducing malicious content into shared tools or data stores

These surfaces require robust malware defenses integrated into agent workflows and runtime environments.

Safeguards

Control 10: Malware Defenses

Safeguard	Title	Description	IG1	IG2	IG3	AI Agent Applicability
10.1	**Deploy and Maintain Anti-Malware Software**	Deploy and maintain anti-malware software on all enterprise assets.	●	●	●	Enforce malware scanning and active content sanitization for all agent retrieval and storage subsystems. Agents act as automated ingestion pipelines, frequently retrieving documents, code, and web content that may contain malware or exploit payloads. Anti-malware defenses must be applied effectively at the point of ingress, scanning all downloads before they are processed. However, scanning alone is insufficient against zero-day exploits targeting the agent's parsing tools (e.g., PDF readers, HTML renderers). Therefore, defenses must include Content Disarm and Reconstruction (CDR) to strip active content, such as malicious JavaScript or embedded macros, before the data enters the agent's reasoning loop. This hygiene must extend to storage, ensuring that vector databases and memory stores do not inadvertently become repositories for dormant malware.
10.2	**Configure Automatic Anti-Malware Signature Updates**	Configure automatic updates for anti-malware signature files on all enterprise assets.	●	●	●	No Additional AI Agent Guidance

Control 10: Malware Defenses

Safeguard	Title	Description	IG1	IG2	IG3	AI Agent Applicability
10.3	**Disable Autorun and Autoplay for Removable Media**	Disable autorun and autoplay auto-execute functionality for removable media.	●	●	●	No Additional AI Agent Guidance
10.4	**Configure Automatic Anti-Malware Scanning of Removable Media**	Configure anti-malware software to automatically scan removable media.		●	●	No Additional AI Agent Guidance
10.5	**Enable Anti-Exploitation Features**	Enable anti-exploitation features on enterprise assets and software, where possible, such as Microsoft® Data Execution Prevention (DEP), Windows® Defender Exploit Guard (WDEG), or Apple® System Integrity Protection (SIP) and Gatekeeper™.		●	●	Enable OS-level anti-exploitation features (e.g., Address Space Layout Randomization (ASLR), Data Execution Prevention (DEP)) on agent hosting servers. This hardens the agent runtime against exploitation of memory corruption vulnerabilities, adding a layer of defense for critical agent infrastructure.
10.6	**Centrally Manage Anti-Malware Software**	Centrally manage anti-malware software.		●	●	If malware is detected, quarantine agent components, including tools, sandboxes, retrieval pipelines, or memory elements, to prevent further spread. Rotate credentials, rebuild runtimes, and isolate affected workloads.
10.7	**Use Behavior-Based Anti-Malware Software**	Use behavior-based anti-malware software.		●	●	Enforce behavioral monitoring on agent runtimes and strictly scan generated code for malicious patterns. Traditional signature-based antivirus often fails to detect the unique threats posed by AI agents, which can dynamically generate malicious code or execute fileless attacks using legitimate system tools. Security controls must employ behavior-based analysis to monitor the agent's execution logic, flagging anomalies such as unexpected shell commands, lateral movement attempts, or the generation of obfuscated scripts. By analyzing the intent of the agent's actions and scanning generated code blocks for known exploit patterns before execution, enterprises can prevent agents from being weaponized as insiders.

Model Hosting and Deployment Considerations

Malware defense requirements differ based on where the agent runs and who controls the runtime.

Deployment Environments

- **Cloud-Hosted Agents:** Use cloud-native malware inspection for serverless runtimes, container registries, file storage systems, and outbound web access. Apply scanning to cloud file ingestion workflows and block malicious content at ingress points.
- **On-Premises or Private Infrastructure Agents:** Apply enterprise anti-malware suites, secure gateway filtering, and host-based defenses to agent servers, memory systems, and retrieval services. Ensure sandboxed execution environments are patched and isolated.
- **Endpoint or Edge Agents:** Local agents must inherit endpoint malware protections, including anti-malware engines, OS-level protections, browser hardening, and file-handling restrictions. Prevent agents from downloading or executing malicious files directly on endpoints.

Runtime Ownership and Control

- **Provider-Managed Runtimes:** Understand which malware defenses the provider applies to agent execution environments. Configure domain restrictions, file filtering, and sandbox policies within provider capabilities. Ensure provider anti-malware coverage is compatible with enterprise requirements.
- **Enterprise-Managed Runtimes:** Enterprises must maintain, update, and monitor malware defenses on all agent components, including sandboxes, containers, retrieval systems, and execution tools. Integrate agent workloads into enterprise malware monitoring systems.
- **Local or Embedded Runtimes:** Apply endpoint malware protections to local runtimes. Prevent local agents from bypassing system-level protections or performing unsafe downloads or executions. Ensure that local agents cannot access or write to sensitive directories without controls.

Additional AI Agent Considerations

- Agents can act as malware amplifiers if attackers feed them malicious content; scanning must occur before content enters memory or downstream systems.
- Code execution tools dramatically increase malware risk; tighten these surfaces by disabling “autorun” (specifically for logic linked to public code repositories) and mandating human-in-the-loop (HITL) approval for high-impact operations.
- Retrieval pipelines importing malicious documents into vector stores can poison downstream reasoning or actions.
- Prohibit the autonomous execution of third-party logic sourced from external repositories, requiring all such code to undergo sandboxed validation and manual authorization before it is permitted to run within the agent environment.

Control 11: Data Recovery

Establish and maintain data recovery practices sufficient to restore in-scope enterprise assets to a pre-incident and trusted state.

AI Agent Applicability

AI agents rely on multiple forms of operational data, such as working memory, long-term memory, vector databases, RAG sources, tool outputs, and session state. Some of this data is transient and should not be backed up (e.g., working memory or sensitive ephemeral state), while other data (e.g., long-term memory stores or organizational embeddings) may be critical for agent function and must be protected with recovery mechanisms.

Agents also generate intermediate artifacts, such as execution outputs, retrieval caches, or synthesized notes, which may be required for forensics or workflow continuity. Recovery planning must distinguish between:

- Data that must be recoverable because it is essential to agent function
- Data that must not persist due to privacy, security, or compliance constraints

Furthermore, since agents may act autonomously, loss or corruption of memory or retrieval pipelines can cause silent misbehavior rather than obvious failures. Therefore, recovery processes must account for both detection and restoration.

Agent Data Relevant to Recovery

Recovery planning for agents should consider:

- **Long-term memory** – summaries, structured state, or knowledge bases
- **Vector stores and embeddings** – organizational embeddings, or retrieval indexes
- **Reference corpora or RAG datasets** – curated documents or structured data stores
- **Tool outputs stored persistently** – files, database entries, or system-generated artifacts
- **Session histories** – when intentionally persisted
- **Audit logs and traces** – required to reconstruct activity
- **Model-generated or agent-generated artifacts** – workflows, drafts, reports, code, or analysis

Recovery mechanisms must ensure integrity, availability, and compliance for these data categories.

Safeguards

Control 11: Data Recovery

Safeguard	Title	Description	IG1	IG2	IG3	AI Agent Applicability
11.1	Establish and Maintain a Data Recovery Process	Establish and maintain a documented data recovery process that includes detailed backup procedures. In the process, address the scope of data recovery activities, recovery prioritization, and the security of backup data. Review and update documentation annually, or when significant enterprise changes occur that could impact this Safeguard.	●	●	●	Define recovery scopes for persistent agent state and artifacts, strictly excluding sensitive ephemeral data. AI Agents rely on persistent state, such as vector indices, long-term memory profiles, and generated tool artifacts, that is critical for operational continuity. Loss of this context forces costly re-indexing or retraining. However, generic backup procedures pose a security risk if they indiscriminately capture everything, including transient, sensitive data like active session tokens, cached credentials, or decrypted PII in working memory. If these are restored, they can bypass data retention policies or reintroduce security vulnerabilities (e.g., restoring a compromised session). Therefore, the recovery process must utilize precise inclusion/exclusion filters to ensure that while the agent's knowledge is preserved, its temporary secrets are not inadvertently resurrected. Note: The technical maintenance of backup systems and storage infrastructure is out of scope; however, this Safeguard is included because the persistence and integrity of the agent's memory are critical to the resilience of the reasoning engine.
11.2	Perform Automated Backups	Perform automated backups of in-scope enterprise assets. Run backups weekly, or more frequently, based on the sensitivity of the data.	●	●	●	Implement backups for critical agent data, including long-term memory stores, vector databases, and retrieval corpora using secure, policy-compliant mechanisms. Ensure backups include required metadata for indexing and retrieval.
11.3	Protect Recovery Data	Protect recovery data with equivalent controls to the original data. Reference encryption or data separation, based on requirements.	●	●	●	Protect backups against unauthorized access. Ensure backup locations are encrypted, access-controlled, and protected from unauthorized read/write operations. Prevent agents themselves from writing to or modifying backup destinations.
11.4	Establish and Maintain an Isolated Instance of Recovery Data	Establish and maintain an isolated instance of recovery data. Example implementations include, version controlling backup destinations through offline, cloud, or off-site systems or services.	●	●	●	Maintain isolated, immutable backups of critical agent memory and configuration data. Protects against ransomware or destructive attacks targeting agent knowledge bases; isolated backups ensure recovery is possible even if the primary environment is compromised.
11.5	Test Data Recovery	Test backup recovery quarterly, or more frequently, for a sampling of in-scope enterprise assets.		●	●	Regularly test restoration procedures for agent memory, retrieval pipelines, and tool-generated artifacts. Confirm that agents function correctly after data recovery, including tasks to verify the semantic integrity of backups related to agent memory, vector stores, and retrieval systems, and that restored data aligns with enterprise policies. Ensure that restoring a backup does not reintroduce poisoned or adversarial content that could hijack agent decision-making or degrade reasoning.

Model Hosting and Deployment Considerations

Deployment Environments

- **Cloud-Hosted Agents:** Use cloud backup services for storage systems supporting long-term memory, vector stores, and RAG corpora. Ensure cross-region replication and backup encryption comply with enterprise recovery policies. Validate that cloud-managed vector or embedding services support backup and restore at required granularity.
- **On-Premises or Private Infrastructure Agents:** Integrate agent data into enterprise backup solutions. Protect backups of databases, file stores, retrieval indexes, and embedding pipelines with internal access controls. Ensure backup windows and restoration procedures match operational requirements for agent workloads.
- **Endpoint or Edge Agents:** Local or embedded agents should not store long-lived memory or retrieval datasets on endpoints unless explicitly required. If persistent data exists locally, ensure endpoint backup policies cover it appropriately. Protect device backups against unauthorized access and avoid restoring sensitive transient state.

Runtime Ownership and Control

- **Provider-Managed Runtimes:** Understand what data the provider retains or backs up by default. For agent memory or retrieval systems managed by a provider, verify retention, backup scheduling, restoration capabilities, and deletion guarantees. Ensure compliance with enterprise data-protection policies.
- **Enterprise-Managed Runtimes:** Enterprises must back up all agent-specific stores, including memory, retrieval systems, indexes, and structured artifacts. Backup procedures must be tested routinely and integrated with CI/CD deployment processes to ensure consistency across environments.
- **Local or Embedded Runtimes:** Avoid persisting sensitive agent data locally. If local persistence is required (e.g., certain offline or edge workflows), ensure backups occur through secure device management systems and that restored data does not violate retention or privacy policies.

Additional AI Agent Considerations

- Corruption of vector stores or retrieval indexes can degrade reasoning silently. Recovery testing must validate semantic correctness, not just byte-level integrity.
- If agent memory contains regulated or sensitive data, backup retention periods must align with Control 3 (Data Protection).
- Backup processes should support versioning to avoid restoring stale or unsafe memory states after rapid development cycles.

Control 12: Network Infrastructure Management

Establish, implement, and actively manage (track, report, correct) network devices, in order to prevent attackers from exploiting vulnerable network services and access points.

AI Agent Applicability

AI agents rely on network pathways to access tools, APIs, model endpoints, retrieval systems, memory stores, browser targets, and other services. Network infrastructure plays a direct role in enforcing the boundaries that govern an agent's capability.

Agents can unintentionally cross trust boundaries, for example, by calling tools in a different environment, accessing unintended network segments, or retrieving external content through corporate networks. Agents may also operate in ephemeral environments (serverless, containers) where network configurations are dynamically created or destroyed.

Network infrastructure must therefore:

- Enforce which systems an agent may reach
- Protect agent runtimes from external scanning or attacks
- Segregate agent memory systems and tools
- Limit outbound web access
- Contain execution tools and sandboxes
- Provide telemetry for monitoring and investigation

Agent Networking Surfaces

Network security for agents must cover:

- **Outbound network access** – tools, APIs, model endpoints, or external URLs the agent may call
- **Inbound access to agent runtimes** – which systems can invoke or trigger agents
- **Internal pathways** – agent communication with memory stores, vector databases, or tool backends
- **Browser or fetch tools** – network controls limiting which domains agents can access
- **Multi-agent communication** – pathways connecting multiple agents or orchestrators
- **Container, serverless, or ephemeral network constructs** – dynamic networking for agent runtimes
- **Model endpoints** – securing connections to LLM/SLM APIs or hosted inference servers

Safeguards

Control 12: Network Infrastructure Management

Safeguard	Title	Description	IG1	IG2	IG3	AI Agent Applicability
12.1	**Ensure Network Infrastructure is Up-to-Date**	Ensure network infrastructure is kept up-to-date. Example implementations include running the latest stable release of software and/or using currently supported network as a service (NaaS) offerings. Review software versions monthly, or more frequently, to verify software support.	●	●	●	Treat software-defined AI networking components, specifically AI Gateways (L7 proxies) and Container Network Interfaces (CNIs), as critical infrastructure. Maintain strict, rapid patch cycles for these layers to mitigate protocol-level vulnerabilities common in these rapidly evolving, open-source network stacks.
12.2	**Establish and Maintain a Secure Network Architecture**	Design and maintain a secure network architecture. A secure network architecture must address segmentation, least privilege, and availability, at a minimum. Example implementations may include documentation, policy, and design components.		●	●	Segment agent runtimes and enforce private, authenticated connectivity for memory and model services. Agents act as the central orchestrator, integrating highly sensitive components like vector databases (memory) and LLM inference endpoints. A secure network architecture must place the agent runtime in a dedicated trust zone, enforcing strict ingress filtering, via ACLs/firewalls, to prevent unauthorized entities from invoking the agent. Furthermore, the connections between the agent and its "brain" (models) or "memory" (retrieval systems) must function over private, authenticated backbones (e.g., PrivateLink, mTLS) rather than public networks. This segmentation ensures that the agent's core logic is shielded from direct internet exposure and protects sensitive retrieval traffic from interception. Furthermore, isolate high-risk execution tools in restricted network sandboxes to prevent lateral movement. Tools that execute dynamic code or interact with the open web (e.g., Python interpreters, headless browsers) inherently introduce high risk. These components must be treated as untrusted and placed in isolated network sandboxes with zero trust access to the internal corporate network. This containment strategy is critical to prevent "breakout" attacks; if an adversary successfully executes malicious code within the interpreter or exploits a browser vulnerability, the strict network isolation ensures they remain trapped in the sandbox, unable to pivot laterally to compromise the host system or inspect internal topology.
12.3	**Securely Manage Network Infrastructure**	Securely manage network infrastructure. Example implementations include version-controlled Infrastructure-as-Code (IaC), and the use of secure network protocols, such as SSH and HTTPS.		●	●	Ensure agents are connecting to all tools and other services using only secure network protocols such as HTTPS. Prevent network tampering and other attacks that can result from using weak network protocols.
12.4	**Establish and Maintain Architecture Diagram(s)**	Establish and maintain architecture diagram(s) and/or other network system documentation. Review and update documentation annually, or when significant enterprise changes occur that could impact this Safeguard.		●	●	Maintain current diagrams of agent network architecture, including data flows to tools and memory stores. Essential for understanding agent connectivity, planning segmentation, and conducting incident response; helps identify unauthorized network paths or dependencies.
12.5	**Centralize Network Authentication, Authorization, and Auditing (AAA)**	Centralize network AAA.		●	●	Centralize network AAA for all agent-to-agent and agent-to-tool communications by integrating agent identities and user roles into network-layer access policies. In multi-agent architectures, enforce zero-trust network segmentation to strictly govern task delegation, state sharing, and data ingestion. Network controls must be context-aware, mapping specific agent RBAC roles to authorized network boundaries, to prevent low-privilege agents from laterally communicating with or prompting high-privilege peers. Furthermore, implement data-aware network filtering to enforce ingress limits, blocking the transmission of data classified above an agent's defined security tier. By ensuring the network layer is informed by the session's privilege context, the enterprise can prevent unauthorized escalation and maintain auditability of all autonomous traffic flows within the agentic environment.

Control 12: Network Infrastructure Management

Safeguard	Title	Description	IG1	IG2	IG3	AI Agent Applicability
12.6	**Use of Secure Network Management and Communication Protocols**	Adopt secure network management protocols (e.g., 802.1X) and secure communication protocols (e.g., Wi-Fi Protected Access 2 (WPA2) Enterprise or more secure alternatives).		●	●	No Additional AI Agent Guidance
12.7	**Ensure Remote Devices Utilize a VPN and are Connecting to an Enterprise's AAA Infrastructure**	Require users to authenticate to enterprise-managed VPN and authentication services prior to accessing enterprise resources on end-user devices.		●	●	No Additional AI Agent Guidance
12.8	**Establish and Maintain Dedicated Computing Resources for All Administrative Work**	Establish and maintain dedicated computing resources, either physically or logically separated, for all administrative tasks or tasks requiring administrative access. The computing resources should be segmented from the enterprise's primary network and not be allowed internet access.			●	Use dedicated admin workstations for managing production agent environments. Reduces the risk of credential theft or malware infection from general-purpose devices compromising high-privilege agent administration accounts.

Model Hosting and Deployment Considerations

Deployment Environments

- **Cloud-Hosted Agents:** Use cloud-native networking constructs such as VPCs, security groups, private endpoints, and service meshes to restrict agent traffic. Limit outbound access to approved model APIs and tools. Ensure ephemeral runtimes (serverless, containers) inherit proper network policies.
- **On-Premises or Private Infrastructure Agents:** Apply internal segmentation using VLANs, access control lists (ACLs), firewalls, and network access control (NAC) policies. Restrict communication to approved databases, retrieval systems, and internal services. Ensure that agent orchestration hosts are not reachable from untrusted networks.
- **Endpoint or Edge Agents:** Agents running locally should inherit device-level firewall rules and network restrictions. Prevent local agents from establishing unauthorized outbound connections or interacting with internal network segments outside their operational scope.

Runtime Ownership and Control

- **Provider-Managed Runtimes:** Understand the networking configuration capabilities provided by the vendor. Configure outbound access controls, restrict reachable domains, and rely on provider-provided segmentation options. Validate that provider-managed sandboxes enforce network isolation.
- **Enterprise-Managed Runtimes:** Enterprises must configure network policies directly, including segmentation, egress rules, access to memory stores, and permitted tool APIs. Ensure container- and serverless-based agents receive consistent networking controls during deployment.
- **Local or Embedded Runtimes:** Local agents rely on endpoint security controls to restrict outbound connections and protect against malicious network activity. Ensure device firewalls and OS-level controls prevent unapproved network interactions.

Additional AI Agent Considerations

- Autonomous agents may attempt unexpected outbound calls; network policies provide a strong safety boundary.
- Execution tools often represent the highest-risk network exposure; ensure that sandboxes cannot reach sensitive internal systems.
- Retrieval pipelines may access internal document stores; segment these systems aggressively.

Control 13: Network Monitoring and Defense

Operate processes and tooling to establish and maintain comprehensive network monitoring and defense against security threats across the enterprise's network infrastructure and user base.

AI Agent Applicability

AI agents introduce new network behaviors because they may autonomously call tools, model endpoints, APIs, retrieval systems, or external URLs. Agents can act as high-frequency consumers of internal services and may unintentionally access systems or domains outside their intended scope. Conversely, agent runtimes may be targeted by attackers seeking to influence agent behavior, exploit tool vulnerabilities, or exfiltrate data through agent-mediated channels.

Since agents can make rapid, multi-step decisions, small misconfigurations or malicious prompts can cause network actions at speed and scale. Continuous monitoring is therefore essential to detect:

- Unexpected outbound API calls
- Suspicious retrieval patterns
- Attempts to access unauthorized internal systems
- Abnormal tool invocation behavior
- High-risk traffic generated autonomously
- Malicious inbound attempts to interact with agent endpoints
- Lateral movement attempts targeting agent runtimes or memory stores

Agent Network Threat Surfaces

Network monitoring for agents must cover:

- Outbound traffic to tools and external APIs
- Outbound access to model endpoints
- Browsing or URL-fetching activity via tools
- Internal traffic to memory stores, vector databases, or retrieval systems
- Agent-to-agent communication in multi-agent workflows
- Inbound events triggering agents (e.g., webhooks, event buses, message queues)
- Network activity inside execution sandboxes

Safeguards

Control 13: Network Monitoring and Defense

Safeguard	Title	Description	IG1	IG2	IG3	AI Agent Applicability
13.1	**Centralize Security Event Alerting**	Centralize security event alerting across enterprise assets for log correlation and analysis. Best practice implementation requires the use of a SIEM, which includes vendor-defined event correlation alerts. A log analytics platform configured with security-relevant correlation alerts also satisfies this Safeguard.		●	●	Correlate network telemetry, audit logs, and tool activity to detect anomalous agent behavior. Agents generate complex footprints across application logs and network layers. To detect compromised agents or malicious workflows effectively, security systems must correlate network telemetry with application audit logs (Control 8). Isolated alerts are often insufficient; true detection requires linking a high-risk tool call or memory access event with its corresponding network traffic to identify anomalies. By centralizing these diverse signals, ranging from unexpected API invocations to policy violations, enterprises can contextualize suspicious behavior and differentiate between benign hallucinations and active exploitation or data exfiltration attempts.
13.2	**Deploy a Host-Based Intrusion Detection Solution**	Deploy a host-based intrusion detection solution on enterprise assets, where appropriate and/or supported.		●	●	Monitor agent runtimes for behavioral anomalies, unauthorized tool usage, and unexpected network connections. Agents operate as autonomous processes that can be hijacked to execute commands or open backdoors. A Host-based Intrusion Detection System (HIDS), or a modern equivalent like EDR, is essential to monitor the agent's internal state. Unlike network firewalls which only see "traffic," a HIDS sees the process creating that traffic. It can detect if an agent process is spawning unexpected shells, modifying critical system files, or initiating unauthorized outbound connections to unknown IP addresses. By correlating these system-level behaviors with network activity, enterprises can identify compromised agents that are attempting lateral movement or data exfiltration from within the trusted host environment.
13.3	**Deploy a Network Intrusion Detection Solution**	Deploy a network intrusion detection solution on enterprise assets, where appropriate. Example implementations include the use of a Network Intrusion Detection System (NIDS) or equivalent cloud service provider (CSP) service.		●	●	Monitor agent network traffic for exploitation, lateral movement, and unauthorized egress. Agents introduce unique traffic patterns that require rigorous inspection via Network Intrusion Detection Systems (NIDS) or cloud-native equivalents. Security teams must configure these systems to detect threats across three specific vectors: Inbound exploitation attempts targeting vulnerable agent frameworks or tool adapters; East-West lateral movement where a compromised agent attempts to scan or probe internal memory systems and databases; and North-South anomalous egress indicating data exfiltration or communication with Command and Control (C2) servers. This network-level visibility is the final backstop to detect when an agent has been successfully weaponized, even if host-level logs are silenced.
13.4	**Perform Traffic Filtering Between Network Segments**	Perform traffic filtering between network segments, where appropriate.		●	●	Enforce traffic filtering between agent runtimes, memory stores, and other internal networks. Prevents lateral movement; if an agent is compromised, filtering restricts it from accessing sensitive systems outside its designated segment.
13.5	**Manage Access Control for Remote Assets**	Manage access control for assets remotely connecting to enterprise resources. Determine amount of access to enterprise resources based on: up-to-date anti-malware software installed, configuration compliance with the enterprise's secure configuration process, and ensuring the operating system and applications are up-to-date.		●	●	No Additional AI Agent Guidance
13.6	**Collect Network Traffic Flow Logs**	Collect network traffic flow logs and/or network traffic to review and alert upon from network devices.		●	●	Continuously monitor network activity from agent runtimes for anomalies and deviations, including unexpected domains, high-risk traffic, unusual request patterns, or repeated failed connections. When agents collaborate, monitor inter-agent traffic for privilege escalation, data exposure, or unintended delegation patterns. Ensure multi-agent routing cannot bypass segmentation controls.

Control 13: Network Monitoring and Defense

Safeguard	Title	Description	IG1	IG2	IG3	AI Agent Applicability
13.7	Deploy a Host-Based Intrusion Prevention Solution	Deploy a host-based intrusion prevention solution on enterprise assets, where appropriate and/or supported. Example implementations include use of an Endpoint Detection and Response (EDR) client or host-based IPS agent.			●	Contain misbehaving or compromised agents. Use runtime controls to pause, isolate, or disable agents exhibiting harmful behavior. Revoke credentials, restrict network access, and stop tool invocation pathways as needed.
13.8	Deploy a Network Intrusion Prevention Solution	Deploy a network intrusion prevention solution, where appropriate. Example implementations include the use of a Network Intrusion Prevention System (NIPS) or equivalent CSP service.			●	Deploy IPS or equivalent cloud-native threat detection and prevention to agent servers, serverless runtimes, containers, and edge devices. Detect and prevent exploitation attempts against tool adapters, browsers, interpreters, or agent orchestration logic. Enable automated or manual containment for agent runtimes that exhibit confirmed malicious activity. Example implementations include blocking outbound traffic, pausing the agent, rotating credentials, or isolating affected compute environments.
13.9	Deploy Port-Level Access Control	Deploy port-level access control. Port-level access control utilizes 802.1x, or similar network access control protocols, such as certificates, and may incorporate user and/or device authentication.			●	No Additional AI Agent Guidance
13.10	Perform Application Layer Filtering	Perform application layer filtering. Example implementations include a filtering proxy, application layer firewall, or gateway.			●	Inspect and filter agent-driven HTTP/S requests by applying content filtering, malware inspection, or proxy-based scanning to HTTP traffic generated by agents, especially agents using browser automation or URL-fetch tools.
13.11	Tune Security Event Alerting Thresholds	Tune security event alerting thresholds monthly, or more frequently.			●	Trigger alerts for abnormal traffic bursts, rapid sequences of tool calls, mass retrievals, or high-volume external requests, as these are indicators of misbehaving or compromised agents.

Model Hosting and Deployment Considerations

Deployment Environments

- **Cloud-Hosted Agents:** Use cloud-native monitoring tools (flow logs, VPC logs, service mesh telemetry, API gateway logs) to track agent traffic. Configure alerts for unexpected egress, unauthorized access attempts, and anomalous inter-service communication. Use cloud firewalls and domain name system (DNS) protections to enforce known-good traffic patterns.
- **On-Premises or Private Infrastructure Agents:** Use enterprise IDS/IPS, network sensors, internal firewalls, and traffic-analysis tools to monitor for unauthorized access to memory stores, retrieval systems, and internal APIs. Ensure visibility into container and orchestrator networking.
- **Endpoint or Edge Agents:** Use device-level firewalls, endpoint detection and response (EDR), local network telemetry, and OS-level protections to monitor outbound connections, suspicious downloads, browser automation activity, or unexpected calls to internal systems.

Runtime Ownership and Control

- **Provider-Managed Runtimes:** Clarify what network telemetry the provider exposes. Configure provider options for monitoring, anomaly detection, and traffic restrictions. Ensure visibility into agent egress and inbound triggers aligns with enterprise monitoring requirements.
- **Enterprise-Managed Runtimes:** Enterprises must implement full monitoring and defense for agent traffic, including scanning, flow analysis, anomaly detection, and blocking. Integrate runtime network events with enterprise SIEM and threat detection tools.
- **Local or Embedded Runtimes:** Local runtimes rely on endpoint monitoring, device-level firewalls, and local threat protection to detect suspicious traffic. Ensure that agents cannot bypass local protections or generate unmonitored outbound connections.

Additional AI Agent Considerations

- Agents can generate high-frequency automated traffic that resembles scanning if misconfigured, and network monitoring must detect this.
- Attackers may attempt to influence agent behavior to exfiltrate data over outbound connections; therefore, tight outbound monitoring is essential.
- Execution tools (e.g., browser, interpreter) may produce network activity independent of agent intent, making it important to monitor these tools separately.

Control 14: Security Awareness and Skills Training

Establish and maintain a security awareness program to influence behavior among the workforce to be security conscious and properly skilled to reduce cybersecurity risks to the enterprise.

AI Agent Applicability

AI agents introduce new operational behaviors, attack surfaces, and failure modes that require targeted training for developers, operators, analysts, and business users. Agents can take actions, access internal systems, and manipulate data in ways traditional applications cannot. Misuse, misconfiguration, or misunderstanding of agent capabilities can lead directly to security incidents.

Training must address:

- How agents work
- The risks introduced by tools, autonomous actions, and reasoning loops
- Proper handling of sensitive data around agents
- How to interact with agents safely
- How to debug or monitor agents without exposing data
- How to identify agent misuse or anomalous behavior
- How to escalate and respond to agent-related incidents

Since AI agents rely on emergent behaviors and dynamic orchestration, human oversight, and well-trained personnel are critical to safety.

Role-Specific Training Needs

Different roles require specialized training:

- **Developers and engineers** – frameworks, tool integration risks, prompt design, memory systems, execution tools, and RAG protection
- **Security teams** – agent threat models, audit logging, monitoring abnormal agent behavior, and incident response for agent-driven events
- **Operators and administrators** – deployment environments, runtime control models, credential handling, and access control management
- **Business users** – safe interaction patterns, data-protection rules, interpreting agent outputs, and escalation procedures
- **Executives and leadership** – governance, risk boundaries, and strategic implications of agent use

Safeguards

Control 14: Security Awareness and Skills Training

Safeguard	Title	Description	IG1	IG2	IG3	AI Agent Applicability
14.1	**Establish and Maintain a Security Awareness Program**	Establish and maintain a security awareness program. The purpose of a security awareness program is to educate the enterprise's workforce on how to interact with enterprise assets and data in a secure manner. Conduct training at hire and, at a minimum, annually. Review and update content annually, or when significant enterprise changes occur that could impact this Safeguard.	●	●	●	Provide training on risks of agent autonomy. Educate personnel about the risks associated with autonomous task execution, recursive loops, and tool-driven chain-of-actions. Emphasize when human review or overrides are required.
14.2	**Train Workforce Members to Recognize Social Engineering Attacks**	Train workforce members to recognize social engineering attacks, such as phishing, business email compromise (BEC), pretexting, and tailgating.	●	●	●	Train users to treat Prompt Injection as social engineering targeting the agent. Users must scrutinize agent outputs for signs that the agent has been tricked by untrusted content into ignoring its instructions. Agents processing external data (e.g., emails, websites) are vulnerable to Indirect Prompt Injection, a form of phishing where the agent, not the human, is the victim, tricking it into executing malicious commands on the user's behalf.
14.3	**Train Workforce Members on Authentication Best Practices**	Train workforce members on authentication best practices. Example topics include MFA, password composition, and credential management.	●	●	●	Train users on the risks of Delegated Authentication, where agents hold long-lived tokens. Emphasize that granting an agent access to a tool (e.g., email) is functionally equivalent to sharing their password with a third party. Users often over-permission agents for convenience. Without understanding that the agent acts with their identity, they may inadvertently authorize an autonomous system to perform irreversible actions (e.g., mass data deletion) without a "human-in-the-loop" check.
14.4	**Train Workforce on Data Handling Best Practices**	Train workforce members on how to identify and properly store, transfer, archive, and destroy sensitive data. This also includes training workforce members on clear screen and desk best practices, such as locking their screen when they step away from their enterprise asset, erasing physical and virtual whiteboards at the end of meetings, and storing data and assets securely.	●	●	●	Reinforce safe interaction patterns for end-users. Train business users on safe ways to interact with agents, including verifying outputs, avoiding over-reliance, understanding limitations, and recognizing when agent recommendations may be unsafe or out of scope.
14.5	**Train Workforce Members on Causes of Unintentional Data Exposure**	Train workforce members to be aware of causes for unintentional data exposure. Example topics include mis-delivery of sensitive data, losing a portable end-user device, or publishing data to unintended audiences.	●	●	●	Educate users that agents ignore Need to Know boundaries unless explicitly constrained. An agent with database access will retrieve all context it deems relevant, potentially surfacing sensitive data the user shouldn't see. Agents lack human discretion, and they often treat all accessible data as context for their reasoning loop, leading to "confused deputy" scenarios where the agent extracts and exposes confidential data to fulfill a benign user request.
14.6	**Train Workforce Members on Recognizing and Reporting Security Incidents**	Train workforce members to be able to recognize a potential incident and be able to report such an incident.	●	●	●	Ensure staff know how to identify suspicious agent behavior or misuse and how to escalate incidents involving agent actions, tool misuse, or data exposure.
14.7	**Train Workforce on How to Identify and Report if Their Enterprise Assets are Missing Security Updates**	Train workforce to understand how to verify and report out-of-date software patches or any failures in automated processes and tools. Part of this training should include notifying IT personnel of any failures in automated processes and tools.	●	●	●	Train staff to recognize agent drift or tool failure as potential security indicators, not just bugs. An agent failing to use a safety tool or bypassing a guardrail may indicate a compromised runtime. In autonomous systems, a missing update isn't just an unpatched OS, it's an agent running an outdated system prompt or deprecated tool definition that lacks current safety constraints, leaving it vulnerable to known jailbreaks.

Control 14: Security Awareness and Skills Training

Safeguard	Title	Description	IG1	IG2	IG3	AI Agent Applicability
14.8	**Train Workforce on the Dangers of Connecting to and Transmitting Enterprise Data Over Insecure Networks**	Train workforce members on the dangers of connecting to, and transmitting data over, insecure networks for enterprise activities. If the enterprise has remote workers, training must include guidance to ensure that all users securely configure their home network infrastructure.	●	●	●	Train users that agents connecting to free or unvetted external tools (e.g., public weather APIs, unverified MCP servers) can expose the agent's internal memory and reasoning trace to third-parties. Agents often send their entire conversation history (i.e., context window) to tools to get a result. Connecting an agent to an insecure or malicious tool endpoint effectively exfiltrates the entire session's sensitive data to that provider.
14.9	**Conduct Role-Specific Security Awareness and Skills Training**	Conduct role-specific security awareness and skills training. Example implementations include secure system administration courses for IT professionals, OWASP® Top 10 vulnerability awareness and prevention training for web application developers, and advanced social engineering awareness training for high-profile roles.		●	●	Conduct tiered training covering agent risks, data safety, runtime hardening, and anomaly detection. Enterprises must implement a tiered training curriculum. All personnel require foundational education on the unique risks of agent autonomy (e.g., "planning" behaviors) and strict data minimization protocols to prevent sensitive data leakage. Beyond these basics, technical roles demand specialized skills. Administrators must be trained on runtime isolation, credential boundaries, and secure configuration of execution tools. Security teams require specific training on interpreting agent telemetry, learning to distinguish benign hallucinations from malicious tool use, and investigating complex incidents involving agent memory or decision logic.

Model Hosting and Deployment Considerations

Deployment Environments

- **Cloud-Hosted Agents:** Training must cover cloud-specific risks, such as exposed endpoints, misconfigured IAM roles, network egress rules, and integration with cloud-native logging and monitoring systems.
- **On-Premises or Private Infrastructure Agents:** Training should include internal platform considerations, segmentation rules, local tool integrations, patching workflows, and the specifics of internal agent hosting frameworks.
- **Endpoint or Edge Agents:** Users and administrators must understand device-level risks, local data exposure, offline behavior, local logging, and safe usage patterns for embedded or locally running agents.

Runtime Ownership and Control

- **Provider-Managed Runtimes:** Training must ensure personnel understand which controls are provider-operated, where configuration boundaries lie, and how to verify provider compliance with enterprise security requirements.
- **Enterprise-Managed Runtimes:** Training should focus on internal responsibilities, secure CI/CD practices, secrets handling, runtime updates, and the safe operation of internally deployed agent frameworks and tools.
- **Local or Embedded Runtimes:** Personnel must understand local-device constraints, including credential storage, local logs, access to files, and the risk that agents may act with local user privileges.

Additional AI Agent Considerations

- Training must evolve as agent patterns, frameworks, and risks evolve.
- Developers should be trained on how agent behavior differs from deterministic software.
- Security staff must understand how to interpret agent logs, policies, and reasoning traces.
- Training should emphasize human oversight, as agents are not inherently trustworthy or safety-aware.

Control 15: Service Provider Management

Develop a process to evaluate service providers who hold sensitive data, or are responsible for an enterprise's critical IT platforms or processes, to ensure these providers are protecting those platforms and data appropriately.

AI Agent Applicability

AI agents depend on a broad ecosystem of third-party services, often far more varied than traditional applications. Examples include:

- Model inference providers
- Embedding and vector database services
- Tool backends accessed via APIs
- Browser automation or content-analysis services
- Hosted agent runtimes
- Cloud-based orchestration or workflow engines
- External data sources used in retrieval pipelines
- Email or file-processing APIs
- SaaS systems invoked through tools or agent logic

Since these providers execute code, store data, perform inference, or supply inputs that agents act upon, providers profoundly affect agent behavior, data exposure, and operational risk. Misconfigured, vulnerable, or overly permissive provider integrations can lead to data leakage, privilege escalation, or unauthorized external actions initiated by the agent.

Service Provider Surfaces Relevant to Agents

When AI agents rely on third-party providers, the following surfaces require scrutiny:

- **Model endpoints** – LLM/SLM inference APIs and associated data handling
- **Tool backends** – MCP tools, API endpoints, execution services, or remote browsers
- **Memory and retrieval services** – hosted vector databases, embedding APIs, or RAG infrastructure
- **Hosted agent runtimes** – fully managed agent services or cloud-native agent platforms
- **Data sources** – external content used by retrieval or enrichment processes
- **Credentials and identities** – secrets or tokens provisioned to providers
- **Telemetry and logs** – data stored or processed by providers as part of monitoring

Managing these risks requires structured oversight.

Safeguards

Control 15: Service Provider Management

Safeguard	Title	Description	IG1	IG2	IG3	AI Agent Applicability
15.1	Establish and Maintain an Inventory of Service Providers	Establish and maintain an inventory of service providers. The inventory is to list all known service providers, include classification(s), and designate an enterprise contact for each service provider. Review and update the inventory annually, or when significant enterprise changes occur that could impact this Safeguard.	●	●	●	Maintain an up-to-date inventory of all external providers used by agents, including model providers, embedding services, tool backends, memory systems, and hosted agent offerings.
15.2	Establish and Maintain a Service Provider Management Policy	Establish and maintain a service provider management policy. Ensure the policy addresses the classification, inventory, assessment, monitoring, and decommissioning of service providers. Review and update the policy annually, or when significant enterprise changes occur that could impact this Safeguard.		●	●	Ensure third-party providers receive access to only required data needed to perform authorized tasks. Prevent agents from sending sensitive or regulated data to providers that lack the necessary protections or authorizations.
15.3	Classify Service Providers	Classify service providers. Classification consideration may include one or more characteristics, such as data sensitivity, data volume, availability requirements, applicable regulations, inherent risk, and mitigated risk. Update and review classifications annually, or when significant enterprise changes occur that could impact this Safeguard.		●	●	Classify agent-related service providers based on the sensitivity of data they process and the criticality of the agent functions they support. Enable risk-based management of providers. For example, high-risk providers (e.g., those hosting core memory or models) receive more scrutiny and stricter controls.
15.4	Ensure Service Provider Contracts Include Security Requirements	Ensure service provider contracts include security requirements. Example requirements may include minimum security program requirements, security incident and/or data breach notification and response, data encryption requirements, and data disposal commitments. These security requirements must be consistent with the enterprise's service provider management policy. Review service provider contracts annually to ensure contracts are not missing security requirements.		●	●	Mandate data isolation, strict retention limits, and incident response Service Level Agreements (SLAs) in service provider contracts. AI agents frequently rely on third-party inference providers and hosted tool execution environments, effectively outsourcing critical logic and memory. Contracts must explicitly govern the entire data life cycle, mandating strict retention limits, secure deletion capabilities, and proven cryptographic isolation to prevent cross-tenant data leakage. Furthermore, agreements must define clear SLAs for incident response, requiring providers to proactively notify the enterprise of breaches and participate in coordinated investigations. This ensures that if a third-party tool or model is compromised, the enterprise retains legal and operational recourse to contain the blast radius rather than being reliant on generic consumer terms.
15.5	Assess Service Providers	Assess service providers consistent with the enterprise's service provider management policy. Assessment scope may vary based on classification(s), and may include review of standardized assessment reports, such as Service Organization Control 2 (SOC 2) and Payment Card Industry (PCI) Attestation of Compliance (AoC), customized questionnaires, or other appropriately rigorous processes. Reassess service providers annually, at a minimum, or with new and renewed contracts.			●	Evaluate provider security posture, focusing on runtime isolation and cross-modality safety controls. Before integrating third-party services into agent workflows, enterprises must conduct rigorous security assessments that extend beyond standard compliance checks. Evaluations must specifically validate the provider's runtime isolation capabilities for tool execution, ensuring sandboxes are truly secure, and ensuring their defense against multimodal attacks. Crucially, if the provider processes non-text inputs (audio, vision), assessments must confirm that safety filters are applied after modality conversion; for example, scanning transcribed text for jailbreaks before it enters the reasoning model. This ensures that third-party integrations do not introduce "backdoor" vulnerabilities where visual or audio inputs can bypass the agent's primary textual safety policies.
15.6	Monitor Service Providers	Monitor service providers consistent with the enterprise's service provider management policy. Monitoring may include periodic reassessment of service provider compliance, monitoring service provider release notes, and dark web monitoring.			●	Monitor agent interactions with third-party services, including model calls, tool requests, retrieval pipelines, and file handling. Detect unexpected or excessive provider usage patterns. Conduct periodic reviews of provider controls, certifications, disclosure history, and architecture changes. Reevaluate risk when providers introduce new features or operational behaviors.

Control 15: Service Provider Management

Safeguard	Title	Description	IG1	IG2	IG3	AI Agent Applicability
15.7	**Securely Decommission Service Providers**	Securely decommission service providers. Example considerations include user and service account deactivation, termination of data flows, and secure disposal of enterprise data within service provider systems.			●	Mandate the verifiable destruction of Derived AI Artifacts, specifically Vector Embeddings, Fine-Tuned Model Weights, and Session Logs, in addition to raw source data. Immediately revoke all federated identities and API keys to prevent the provider's runtime from retaining "zombie access" to enterprise tools. Note: Although model fine-tuning processes are out of scope for this guide, the management and destruction of the resulting model weights are included here to ensure complete decommissioning of the service provider relationship.

Model Hosting and Deployment Considerations

Deployment Environments

- **Cloud-Hosted Agents:** Cloud-hosted agents commonly depend on cloud-native provider services. Ensure cloud vendors' tool backends, model endpoints, and vector stores meet enterprise security requirements. Use private networking, VPC endpoints, or zero-trust controls when possible.
- **On-Premises or Private Infrastructure Agents:** Even when agents run on private infrastructure, they may call external model providers or SaaS APIs. Validate provider controls rigorously, restrict outbound access, and ensure retrieval or tool traffic aligns with security policy.
- **Endpoint or Edge Agents:** Local agents may interact with cloud provider APIs for models or tools. Validate the safety of remote services accessed by embedded agents and ensure endpoint policies prevent uncontrolled communication with external providers.

Runtime Ownership and Control

- **Provider-Managed Runtimes:** For hosted agent runtimes, evaluate provider controls around tool invocation, logging, memory isolation, credential handling, and outbound communications. Confirm the provider enforces least privilege and provides visibility necessary for auditing.
- **Enterprise-Managed Runtimes:** Enterprises must ensure any third-party services integrated into their own agent runtimes meet internal risk criteria. Integrate provider evaluation into CI/CD, architecture review, and runtime deployment decisions.
- **Local or Embedded Runtimes:** Local agents may use third-party APIs directly from devices. Ensure that these providers are vetted and that local agents cannot accidentally leak sensitive data to unapproved services.

Additional AI Agent Considerations

- Providers offering model inference, embeddings, or tool execution often store metadata; ensure this metadata does not violate data-protection policies.
- Integration with external tools or SaaS systems expands the agent blast radius; provider selection should consider tool scope and privilege implications.
- Retrieval pipelines may use external data-enrichment services; ensure these services do not introduce unsafe or poisoned content into memory or RAG systems.

Control 16: Application Software Security

Manage the security life cycle of in-house developed, hosted, or acquired software to prevent, detect, and remediate security weaknesses before they can impact the enterprise.

AI Agent Applicability

AI agents are applications composed of orchestration logic, model-integration layers, tool interfaces, memory systems, retrieval pipelines, and (often) autonomous execution loops. Unlike traditional applications, agents:

- Develop behavior dynamically through model reasoning
- Invoke external tools and APIs based on their own decisions
- Process untrusted content from retrieval systems
- Generate code, file operations, or browser actions
- Maintain internal working memory that influences outputs
- May chain actions recursively or autonomously
- Interact with sensitive internal systems

Agents must therefore be designed and implemented using rigorous secure-development principles. Testing, validation, guardrails, and runtime constraints are essential to ensure that agents do not misbehave, leak data, or perform unsafe actions.

Agent-Specific Application Security Surfaces

Agent application security must consider:

- **Prompt and policy configuration** – system prompts, routing prompts, or tool instructions
- **Tool invocation logic** – safe design of tool handlers, parameter validation, or output sanitization
- **Memory management** – creating, reading, writing, and updating working or long-term memory
- **Retrieval pipelines and RAG sources** – ingestion of untrusted or poisoned documents
- **Execution tools** – code interpreters, sandboxed shells, browser tools, or file handlers
- **Model interactions** – inputs, outputs, and model-side constraints (covered in the *AI LLM Companion Guide*)
- **Multi-agent coordination** – delegation, messaging, shared memory, and consensus mechanisms
- **Orchestration frameworks** – state machines, planners, decision loops, and error-handling paths

Successful application security for agents must integrate these factors into both development and operational patterns.

Safeguards

Control 16: Application Software Security

Safeguard	Title	Description	IG1	IG2	IG3	AI Agent Applicability
16.1	**Establish and Maintain a Secure Application Development Process**	Establish and maintain a secure application development process. In the process, address such items as: secure application design standards, secure coding practices, developer training, vulnerability management, security of third-party code, and application security testing procedures. Review and update documentation annually, or when significant enterprise changes occur that could impact this Safeguard.		●	●	Implement secure deployment pipelines for agents by using CI/CD pipelines that enforce code quality checks, dependency scanning, environment hardening, and secret stripping before deployment. Ensure agent updates follow controlled release processes.
16.2	**Establish and Maintain a Process to Accept and Address Software Vulnerabilities**	Establish and maintain a process to accept and address reports of software vulnerabilities, including providing a means for external entities to report. The process is to include such items as: a vulnerability handling policy that identifies reporting process, responsible party for handling vulnerability reports, and a process for intake, assignment, remediation, and remediation testing. As part of the process, use a vulnerability tracking system that includes severity ratings and metrics for measuring timing for identification, analysis, and remediation of vulnerabilities. Review and update documentation annually, or when significant enterprise changes occur that could impact this Safeguard. Third-party application developers need to consider this an externally-facing policy that helps to set expectations for outside stakeholders.		●	●	Assess vulnerabilities in tooling and execution surfaces. Evaluate the security of tools available to agents – such as code interpreters, browser tools, shell environments, or MCP tools – and remediate vulnerabilities quickly. Tools often present the highest-risk execution surfaces for agents.
16.3	**Perform Root Cause Analysis on Security Vulnerabilities**	Perform root cause analysis on security vulnerabilities. When reviewing vulnerabilities, root cause analysis is the task of evaluating underlying issues that create vulnerabilities in code, and allows development teams to move beyond just fixing individual vulnerabilities as they arise.		●	●	Conduct root cause analysis for vulnerabilities discovered in agent code or configurations. Identifying the underlying cause of vulnerabilities (e.g., insecure prompt design, unsafe tool handling) helps prevent recurrence and improves the overall security of agent development practices.
16.4	**Establish and Manage an Inventory of Third-Party Software Components**	Establish and manage an updated inventory of third-party components used in development, often referred to as a "bill of materials," as well as components slated for future use. This inventory is to include any risks that each third-party component could pose. Evaluate the list at least monthly to identify any changes or updates to these components, and validate that the component is still supported.		●	●	Maintain a detailed inventory of all third-party libraries and components used in agent applications (e.g., AI-BOM). Enables rapid identification of vulnerable components (e.g., a flawed vector store client) and facilitates effective patch management and risk assessment.
16.5	**Use Up-to-Date and Trusted Third-Party Software Components**	Use up-to-date and trusted third-party software components. When possible, choose established and proven frameworks and libraries that provide adequate security. Acquire these components from trusted sources or evaluate the software for vulnerabilities before use.		●	●	Validate software integrity for agent components. Ensure agent frameworks, tool adapters, retrieval modules, and model clients are sourced from trusted repositories and validated through checksums, signatures, or supply-chain security controls. Apply software provenance checks where supported.
16.6	**Establish and Maintain a Severity Rating System and Process for Application Vulnerabilities**	Establish and maintain a severity rating system and process for application vulnerabilities that facilitates prioritizing the order in which discovered vulnerabilities are fixed. This process includes setting a minimum level of security acceptability for releasing code or applications. Severity ratings bring a systematic way of triaging vulnerabilities that improves risk management and helps ensure the most severe bugs are fixed first. Review and update the system and process annually.		●	●	Establish a severity rating system for agent-specific vulnerabilities, considering factors like autonomy and tool access. This helps prioritize remediation efforts by identifying the most critical risks to agent security and operations, ensuring resources are focused on the most dangerous flaws.
16.7	**Use Standard Hardening Configuration Templates for Application Infrastructure**	Use standard, industry-recommended hardening configuration templates for application infrastructure components. This includes underlying servers, databases, and web servers, and applies to cloud containers, Platform as a Service (PaaS) components, and SaaS components. Do not allow in-house developed software to weaken configuration hardening.		●	●	No Additional AI Agent Guidance

Control 16: Application Software Security

Safeguard	Title	Description	IG1	IG2	IG3	AI Agent Applicability
16.8	**Separate Production and Non-Production Systems**	Maintain separate environments for production and non-production systems.		●	●	No Additional AI Agent Guidance
16.9	**Train Developers in Application Security Concepts and Secure Coding**	Ensure that all software development personnel receive training in writing secure code for their specific development environment and responsibilities. Training can include general security principles and application security standard practices. Conduct training at least annually and design in a way to promote security within the development team, and build a culture of security among the developers.		●	●	Train developers on secure agent design and tool use. Ensure developers understand safe tool integration, prompt and policy design, memory handling, RAG boundaries, execution tool risks, and how to avoid embedding sensitive data in prompts or memory.
16.10	**Apply Secure Design Principles in Application Architectures**	Apply secure design principles in application architectures. Secure design principles include the concept of least privilege and enforcing mediation to validate every operation that the user makes, promoting the concept of "never trust user input." Examples include ensuring that explicit error checking is performed and documented for all input, including for size, data type, and acceptable ranges or formats. Secure design also means minimizing the application infrastructure attack surface, such as turning off unprotected ports and services, removing unnecessary programs and files, and renaming or removing default accounts.		●	●	Harden agent orchestration logic by applying standard application security testing (including SAST, DAST, and dependency scanning) to the framework while addressing unique cognitive vulnerabilities. Implement strict circuit breakers to detect and terminate recursion or runaway action chains, such as an agent endlessly calling the same tool, and design multi-agent architectures to prevent "confused deputy" attacks where low-privilege agents manipulate high-privilege peers into executing unauthorized tasks. The system must maintain secure failure modes by failing "closed," ensuring that model timeouts or tool errors terminate the session safely without exposing stack traces or falling back to insecure defaults. Establish rigorous hygiene at every interface by validating all tool inputs and outputs to prevent shell injection or the poisoning of the agent's working memory. Enforce regex-based scrubbing to ensure secrets and credentials are never written to logs, memory, or user outputs. Treat multimodal inputs, including images and audio, as highly untrusted, requiring transcription and safety filtering to detect visual jailbreaks before content enters the reasoning loop. Similarly, protect retrieval pipelines by scanning and sanitizing documents before indexing them into vector stores to prevent knowledge poisoning, where malicious content alters the agent's future decision-making behavior. Mandate deterministic defense-in-depth by isolating dangerous capabilities (specifically code interpreters, shell tools, and browser automation) in ephemeral, network-restricted sandboxes. This runtime isolation must be supported by capability stripping, where unused system calls and network ports are hard-disabled to minimize the attack surface. Finally, implement a deterministic policy enforcement layer, or guardrail middleware, to intercept every agent action before execution. This layer must validate tool calls against static policies, such as "Allow Read" or "Deny Write," and block high-risk requests regardless of the agent's intent or generated reasoning.
16.11	**Leverage Vetted Modules or Services for Application Security Components**	Leverage vetted modules or services for application security components, such as identity management, encryption, auditing, and logging. Using platform features in critical security functions will reduce developers' workload and minimize the likelihood of design or implementation errors. Modern operating systems provide effective mechanisms for identification, authentication, and authorization and make those mechanisms available to applications. Use only standardized, currently accepted, and extensively reviewed encryption algorithms. Operating systems also provide mechanisms to create and maintain secure audit logs.		●	●	Use vetted, standard security modules for agent authentication, encryption, and input validation. This avoids rolling your own security crypto or logic, which is prone to errors. Vetted modules provide a higher level of assurance and reduce the risk of implementation flaws.
16.12	**Implement Code-Level Security Checks**	Apply static and dynamic analysis tools within the application life cycle to verify that secure coding practices are being followed.			●	Implement static and dynamic code analysis (SAST/DAST) in the agent development pipeline. Detects security flaws in agent code (e.g., insecure tool calls, hardcoded secrets) early in the development life cycle, reducing the cost and risk of remediation.

Control 16: Application Software Security

Safeguard	Title	Description	IG1	IG2	IG3	AI Agent Applicability
16.13	**Conduct Application Penetration Testing**	Conduct application penetration testing. For critical applications, authenticated penetration testing is better suited to finding business logic vulnerabilities than code scanning and automated security testing. Penetration testing relies on the skill of the tester to manually manipulate an application as an authenticated and unauthenticated user.			●	Test agent workflows using automated tests, adversarial inputs, evaluation harnesses, and red-team scenarios. Validate that agents behave safely even with malformed inputs or unexpected outputs.
16.14	**Conduct Threat Modeling**	Conduct threat modeling. Threat modeling is the process of identifying and addressing application security design flaws within a design, before code is created. It is conducted through specially trained individuals who evaluate the application design and gauge security risks for each entry point and access level. The goal is to map out the application, architecture, and infrastructure in a structured way to understand its weaknesses.			●	Perform threat modeling for agent systems, considering unique threats like prompt injection, goal misalignment, and tool abuse. This proactively identifies security design flaws and attack vectors specific to agents, enabling the implementation of effective mitigations before deployment.

Model Hosting and Deployment Considerations

Deployment Environments

- **Cloud-Hosted Agents:** Leverage cloud-native security scanning (for functions, containers, serverless tasks), enforce secure API integration, and apply IAM boundaries that limit agent access. Ensure retrieval pipelines, memory stores, and vector databases in the cloud follow secure deployment patterns.
- **On-Premises or Private Infrastructure Agents:** Apply internal Software Development Life Cycle (SDLC) security standards to all agent components, such as frameworks, retrieval systems, model clients, and tool adapters. Ensure on-premises execution tools are sandboxed and updated.
- **Endpoint or Edge Agents:** Local agents must rely on secure installation, OS hardening, application sandboxing, and restricted execution environments. Validate local agent updates before deployment to endpoints.

Runtime Ownership and Control

- **Provider-Managed Runtimes:** Confirm the provider's secure development practices for hosted agent runtimes, including patching, dependency management, isolation, and guardrail enforcement. Validate safe defaults for tool use and memory handling.
- **Enterprise-Managed Runtimes:** Enterprises must implement secure development and deployment pipelines for agent code. Apply strong configuration management, policy layers, and rigorous testing of custom tools or retrieval components.
- **Local or Embedded Runtimes:** Ensure agent code and dependencies deployed to endpoints are signed, verified, and securely updated. Use OS sandboxing features to protect against unsafe model outputs or tool usage triggered locally.

Additional AI Agent Considerations

- Retrieval poisoning can cause subtle misbehavior in agents; development pipelines must validate ingest sources and sanitize content.
- Execution tools drastically expand the attack surface; treat them as untrusted code-execution surfaces requiring stricter controls than typical app modules.
- Policy-enforcement layers should intercept tool calls before execution, not rely solely on prompts.
- Testing must simulate real-world prompts, tool outputs, retrieval failures, and misleading content, incorporating automated red teaming to simulate adversarial prompts, because agent behavior is emergent, not predetermined.

Control 17: Incident Response Management

Establish a program to develop and maintain an incident response capability (e.g., policies, plans, procedures, defined roles, training, and communications) to prepare, detect, and quickly respond to an attack.

AI Agent Applicability

AI agents introduce new categories of incidents, new failure modes, and new investigation requirements. Agents may:

- Execute unauthorized tool calls
- Perform unintended multi-step actions
- Access or exfiltrate sensitive data
- Propagate malicious inputs (e.g., poisoned retrieval content)
- Misinterpret ambiguous prompts or instructions
- Reflect adversarial inputs into harmful behavior
- Execute malicious or incorrect code
- Interact with external systems in unintended ways
- Make decisions based on corrupted memory or poisoned index entries

Since agents operate at speed, can chain together actions, and can interact with many systems, they can cause faster and broader-reaching incidents than traditional applications. Incident response processes must therefore incorporate agent-specific investigation, containment, and recovery steps.

Agent-Related Incident Categories

Typical agent-related incident patterns include:

- **Tool misuse** – unauthorized API calls, unsafe file operations, or dangerous code execution
- **Data leakage incidents** – accidental or adversarial exposure of sensitive input, memory, or tool responses
- **Autonomy failures** – runaway loops, repeated harmful actions, or unintended multi-step chains
- **RAG poisoning or memory corruption** – malicious or inaccurate content injected into corpora, vector stores, or long-term memory
- **Model-manipulation incidents** – adversarial prompts leading to unsafe agent decisions
- **Cross-agent escalation** – unintended behavior arising from multi-agent workflows
- **Provider-side incidents** – breaches or failures in third-party model or tool providers
- **Misconfiguration incidents** – incorrect prompts, tool definitions, or policy settings leading to unsafe behavior

Incident response must be prepared for these unique patterns.

Safeguards

Control 17: Incident Response Management

Safeguard	Title	Description	IG1	IG2	IG3	AI Agent Applicability
17.1	**Designate Personnel to Manage Incident Handling**	Designate one key person, and at least one backup, who will manage the enterprise's incident handling process. Management personnel are responsible for the coordination and documentation of incident response and recovery efforts and can consist of employees internal to the enterprise, service providers, or a hybrid approach. If using a service provider, designate at least one person internal to the enterprise to oversee any third-party work. Review annually, or when significant enterprise changes occur that could impact this Safeguard.	●	●	●	Designate personnel with specific expertise in AI agents and their failure modes to manage agent-related incidents. Agent incidents require specialized knowledge (e.g., understanding autonomy loops, tool logs). Designated experts ensure a rapid and effective response to complex AI failures.
17.2	**Establish and Maintain Contact Information for Reporting Security Incidents**	Establish and maintain contact information for parties that need to be informed of security incidents. Contacts may include internal staff, service providers, law enforcement, cyber insurance providers, relevant government agencies, Information Sharing and Analysis Center (ISAC) partners, or other stakeholders. Verify contacts annually to ensure that information is up-to-date.	●	●	●	Maintain contact info for reporting agent incidents to internal teams and external providers (e.g., model/tool vendors). This ensures rapid communication and coordination during an incident, enabling quicker containment and remediation of agent-related issues.
17.3	**Establish and Maintain an Enterprise Process for Reporting Incidents**	Establish and maintain a documented enterprise process for the workforce to report security incidents. The process includes reporting timeframe, personnel to report to, mechanism for reporting, and the minimum information to be reported. Ensure the process is publicly available to all of the workforce. Review annually, or when significant enterprise changes occur that could impact this Safeguard.	●	●	●	Define a process for reporting suspected agent misbehavior, autonomy failures, or unexpected tool usage. Encourages timely reporting of anomalies by users and operators, allowing security teams to detect and respond to agent incidents early in the kill chain.
17.4	**Establish and Maintain an Incident Response Process**	Establish and maintain a documented incident response process that addresses roles and responsibilities, compliance requirements, and a communication plan. Review annually, or when significant enterprise changes occur that could impact this Safeguard.		●	●	Integrate agent-specific threat scenarios and remediation protocols into enterprise incident response plans. Traditional incident response playbooks often fail to address the speed and autonomy of AI agents. Enterprises must update IR frameworks to explicitly cover agent-specific attack vectors, such as prompt injection, hallucinatory data leakage, recursive loop exhaustion, and vector store poisoning. These scenarios must be integrated into the central enterprise incident management system to ensure consistent tracking, escalation, and regulatory reporting. Crucially, the process must define specialized remediation steps for non-deterministic systems, specifically purging poisoned embeddings from memory stores, rotating compromised tool credentials, and rebuilding tainted runtime environments, ensuring that responders can rapidly contain "runaway" agents and restore trusted states.
17.5	**Assign Key Roles and Responsibilities**	Assign key roles and responsibilities for incident response, including staff from legal, IT, information security, facilities, public relations, human resources, incident responders, analysts, and relevant third parties. Review annually, or when significant enterprise changes occur that could impact this Safeguard.		●	●	Define roles and responsibilities for detecting, triaging, containing, and remediating agent-driven incidents. Ensure that development, security, and operations teams understand their duties during agent-specific events.
17.6	**Define Mechanisms for Communicating During Incident Response**	Determine which primary and secondary mechanisms will be used to communicate and report during a security incident. Mechanisms can include phone calls, emails, secure chat, or notification letters. Keep in mind that certain mechanisms, such as emails, can be affected during a security incident. Review annually, or when significant enterprise changes occur that could impact this Safeguard.		●	●	No Additional AI Agent Guidance
17.7	**Conduct Routine Incident Response Exercises**	Plan and conduct routine incident response exercises and scenarios for key personnel involved in the incident response process to prepare for responding to real-world incidents. Exercises need to test communication channels, decision making, and workflows. Conduct testing on an annual basis, at a minimum.		●	●	No Additional AI Agent Guidance

Control 17: Incident Response Management

Safeguard	Title	Description	IG1	IG2	IG3	AI Agent Applicability
17.8	**Conduct Post-Incident Reviews**	Conduct post-incident reviews. Post-incident reviews help prevent incident recurrence through identifying lessons learned and follow-up action.		●	●	Investigate root causes for agent incidents. Analyze the chain of events leading to the incident, including tool outputs, memory state, retrieval content, reasoning traces, prompt configurations, and policy decisions. Determine whether issues stemmed from an agent system-level issue, such as malicious input, misconfiguration, or corrupted memory, or from larger control framework issues such as gaps in monitoring, policy enforcement, development processes, tool design, or model behavior. Integrate lessons learned into future guardrails, prompts, and agent architectures
17.9	**Establish and Maintain Security Incident Thresholds**	Establish and maintain security incident thresholds, including, at a minimum, differentiating between an incident and an event. Examples can include: abnormal activity, security vulnerability, security weakness, data breach, privacy incident, etc. Review annually, or when significant enterprise changes occur that could impact this Safeguard.			●	Define thresholds for agent behavior (e.g., API rate limits, cost spikes) that trigger incident response actions. Enables automated detection and response to anomalous agent activity, ensuring that runaway or compromised agents are identified and contained quickly.

Model Hosting and Deployment Considerations

Deployment Environments

- **Cloud-Hosted Agents:** Ensure cloud-native logs, flow logs, and agent telemetry are available for investigation. Enable controls to isolate misbehaving serverless functions, containers, or managed agent runtimes. Confirm incident response integration with cloud provider incident notification channels.
- **On-Premises or Private Infrastructure Agents:** Ensure local logging, retrieval systems, memory stores, and execution sandboxes produce sufficient forensic data. Provide mechanisms to isolate agent workloads on internal compute platforms (container orchestrators, clusters, or dedicated servers).
- **Endpoint or Edge Agents:** For local or embedded agents, ensure endpoint logs and telemetry capture agent actions, external requests, and tool outputs. Use endpoint security controls to quarantine compromised local runtimes.

Runtime Ownership and Control

- **Provider-Managed Runtimes:** Understand provider-side incident response processes and what evidence is available during provider-managed incidents. Ensure providers support pausing, disabling, or isolating hosted agents. Validate procedures for handling breaches or failures in provider tool backends or model endpoints.
- **Enterprise-Managed Runtimes:** Enterprises must be able to isolate agent workers, control network access, reset state, rotate credentials, rebuild memory stores, and shut down compromised components. Ensure CI/CD pipelines can rapidly deploy remediations.
- **Local or Embedded Runtimes:** Device-level containment is critical: revoke network access, disable local tools, clear sensitive memory artifacts, and coordinate with endpoint security teams to remediate infected or compromised devices.

Additional AI Agent Considerations

- Agent incidents often require reconstruction of reasoning traces, which are not typically part of standard application forensics.
- Retrieval poisoning incidents require restoring RAG stores or vector databases from clean backups (Control 11). Note: The act of restoring a database is an infrastructure function; however, it is highlighted here because the integrity of the agent's memory is the primary behavioral dependency for safe recovery.
- Tool misuse incidents may require reviewing tool definitions and access controls (Control 6).
- Developers must be involved in investigations because agent behavior depends on configuration, prompts, and tool definitions as much as code.
- Agent autonomy and recursive workflows can cause rapid escalation; isolation and containment must be fast and reliable.

Control 18: Penetration Testing

Test the effectiveness and resiliency of enterprise assets through identifying and exploiting weaknesses in controls (people, processes, and technology), and simulating the objectives and actions of an attacker.

AI Agent Applicability

AI agents introduce nontraditional attack surfaces that must be included in penetration testing and red-team operations. Unlike conventional software, agents can be manipulated through:

- Adversarial prompts
- Crafted tool outputs
- Poisoned retrieval content
- Manipulated memory entries
- Hostile model responses
- Emergent behavior from multi-step reasoning
- Unsafe multi-agent interactions
- Misconfigured policies or tool definitions

Penetration testing must therefore go beyond source-code review or endpoint testing. It must evaluate agent reasoning resilience, tool-call safety, retrieval hardening, memory integrity, and policy enforcement.

Properly designed adversarial testing uncovers vulnerabilities in:

- Agent logic and decision boundaries
- Planning and reasoning loops
- Guardrail or policy bypass
- User-to-agent and agent-to-tool trust boundaries
- Code execution tools
- Browser or file-handling tools
- Model–agent interactions
- Multi-agent coordination
- Vector-store and RAG ingestion paths

Testing must reflect real-world threats: adversarial content, malicious instructions, contaminated retrieval sources, malformed tool responses, and misaligned agent autonomy.

.

Safeguards

CIControl 18: Penetration Testing

Safeguard	Title	Description	IG1	IG2	IG3	AI Agent Applicability
18.1	**Establish and Maintain a Penetration Testing Program**	Establish and maintain a penetration testing program appropriate to the size, complexity, industry, and maturity of the enterprise. Penetration testing program characteristics include scope, such as network, web application, Application Programming Interface (API), hosted services, and physical premise controls; frequency; limitations, such as acceptable hours, and excluded attack types; point of contact information; remediation, such as how findings will be routed internally; and retrospective requirements.		●	●	Explicitly include AI-agent components, such as tools, memory systems, retrieval pipelines, orchestration logic, and configuration, in penetration testing and red-team exercises. Ensure tests cover both direct and indirect manipulation pathways.
18.2	**Perform Periodic External Penetration Tests**	Perform periodic external penetration tests based on program requirements, no less than annually. External penetration testing must include enterprise and environmental reconnaissance to detect exploitable information. Penetration testing requires specialized skills and experience and must be conducted through a qualified party. The testing may be clear box or opaque box.		●	●	Test execution and sandbox surfaces. Assess sandbox isolation, file-system controls, and network restrictions for execution tools (interpreters, shells, browser automation). Attempt code injection, environment breakout, and malicious script execution. If the agent accepts non-text inputs (e.g., screen reading, file uploads, audio commands), use adversarial images (e.g., visual jailbreaks, steganographic commands) or audio to test whether safety guardrails can be bypassed. Verify that the agent does not execute unsafe commands hidden in screenshots or uploaded documents.
18.3	**Remediate Penetration Test Findings**	Remediate penetration test findings based on the enterprise's documented vulnerability remediation process. This should include determining a timeline and level of effort based on the impact and prioritization of each identified finding.		●	●	Remediate findings and update agent design. Address vulnerabilities found during agent penetration tests, such as improving prompts, tool definitions, memory protections, retrieval pipelines, and policy enforcement layers. Incorporate findings into future development cycles.
18.4	**Validate Security Measures**	Validate security measures after each penetration test. If deemed necessary, modify rulesets and capabilities to detect the techniques used during testing.			●	Ensure security teams can detect and investigate malicious agent activity during testing, including anomalous tool calls, data access, memory writes, outbound connections, or execution events.
18.5	**Perform Periodic Internal Penetration Tests**	Perform periodic internal penetration tests based on program requirements, no less than annually. The testing may be clear box or opaque box.			●	Simulate adversarial prompts, memory poisoning, and autonomous workflow exploits to validate reasoning limits, noting that standard penetration tests often miss an agent's unique cognitive vulnerabilities. Internal testing must target the reasoning engine using Red Team scenarios that simulate adversarial prompts, ambiguous instructions, and recursive logic traps designed to force policy bypasses. Testers must attack the agent's memory by injecting conflicting or malicious content into retrieval corpora, such as RAG poisoning, to verify if the agent hallucinates or acts on untrusted data. Beyond the prompt, testers must treat tool interfaces, including Python interpreters, API connectors, and browser drivers, as hostile entry points. Tests should attempt to manipulate API parameters to force unauthorized tool calls, escape sandboxes, or escalate privileges within the execution environment. This includes assessing integration boundaries between the runtime and model endpoints to ensure the agent cannot be coerced into performing unauthorized file system operations, network scans, or "confused deputy" attacks that exceed its permission scope. Finally, testing must stress-test autonomy by attempting to trigger unbounded loops or multi-step chains, ensuring behavioral guardrails effectively arrest unsafe decision paths before they spiral out of control.

Model Hosting and Deployment Considerations

Deployment Environments

- **Cloud-Hosted Agents:** Penetration tests must include cloud-specific surfaces: serverless runtimes, container isolation, network segmentation, identity and access management (IAM) misconfigurations, cloud tool backends, and cloud-hosted vector stores. Validate egress restrictions, private endpoints, and cloud-native guardrails.
- **On-Premises or Private Infrastructure Agents:** Test internal network segmentation, retrieval-system isolation, API access controls, and sandbox configurations. Assess enterprise-managed execution tools, memory systems, and agent frameworks running on private compute.
- **Endpoint or Edge Agents:** Penetration tests must examine local sandboxing, device-level permissions, local file-system access, and outbound connection controls. Test how local agents behave under malicious prompts or untrusted local files.

Runtime Ownership and Control

- **Provider-Managed Runtimes:** Testing must evaluate provider-exposed configuration boundaries, tool restrictions, and policy layers. Validate that provider guardrails cannot be bypassed and that provider APIs enforce safe defaults. Understand provider restrictions on penetration testing and coordinate accordingly.
- **Enterprise-Managed Runtimes:** Test all internally built components, such as frameworks, orchestration logic, tool definitions, retrieval pipelines, vector stores, execution sandboxes, and custom model clients. Validate deployment pipelines and configuration controls.
- **Local or Embedded Runtimes:** Penetration testing should evaluate local agent behavior using malicious files, adversarial prompts, and unexpected tool outputs. Ensure device-level protections limit agent behavior.

Additional AI Agent Considerations

- Penetration testing must include prompt injection, adversarial retrieval content, and malformed tool outputs, none of which exist in traditional apps.
- RAG poisoning simulations are essential to test memory and retrieval defenses.
- Execution tools (code interpreters, shell tools, browsers) require separate red-team treatment because they introduce high-risk behaviors not typically part of application security.
- Multi-agent systems must be tested for unsafe delegation, cross-agent privilege escalation, and shared-memory manipulation.
- Testing should be iterative and repeated as agent behavior, tools, and models evolve.

Conclusion

As enterprises accelerate their adoption of AI agents and LLM-driven workflows, it becomes increasingly important to anchor these emerging capabilities in established, battle-tested security frameworks. The CIS Controls provide precisely that foundation, offering a structured, prioritized approach to asset management, identity governance, secure configuration, monitoring, and incident readiness. By interpreting these controls through the lens of agent behavior, orchestration, tool use, retrieval, and autonomy, security teams can extend familiar Safeguards to an environment where cognitive vulnerabilities, emergent behaviors, and novel attack surfaces now coexist alongside traditional ones. This alignment ensures that even as AI systems grow more capable, their underlying design, operation, and integration adhere to the same security discipline expected of any critical enterprise technology.

Ultimately, securing AI agents is not about inventing an entirely new governance model, but about applying the CIS Controls with deeper awareness of how reasoning engines, memory systems, and autonomous workflows reshape operational risk. Aligning AI agent security to the CIS Controls enables enterprises to innovate responsibly, maintaining agility while ensuring that safety, reliability, and resilience remain core to every AI-powered capability they deploy.

Appendix A: CIS Controls

The CIS Critical Security Controls® (CIS Controls®) are a prioritized set of actions which collectively form a defense-in-depth set of best practices that mitigate the most common attacks against systems and networks. They are developed by a community of information technology (IT) experts who apply their first-hand experience as cyber defenders to create these globally accepted security best practices. The experts who develop the CIS Controls come from a wide range of sectors, including retail, manufacturing, healthcare, education, government, defense, and others. It is important to note that while the CIS Controls address general best practices that enterprises should implement to protect their environment, some operational environments may present unique requirements not addressed by the CIS Controls or require deviations from best practices.

Implementation Groups

The Implementation Group methodology was developed as a new way to prioritize the CIS Controls. These IGs provide a simple and accessible way to help enterprises of different classes focus their scarce security resources, while still leveraging the value of the CIS Controls program, community, and complementary tools and working aids. More about the Implementation Groups can be found in our Guide to Implementation Groups (IG): CIS Critical Security Controls v8.1.

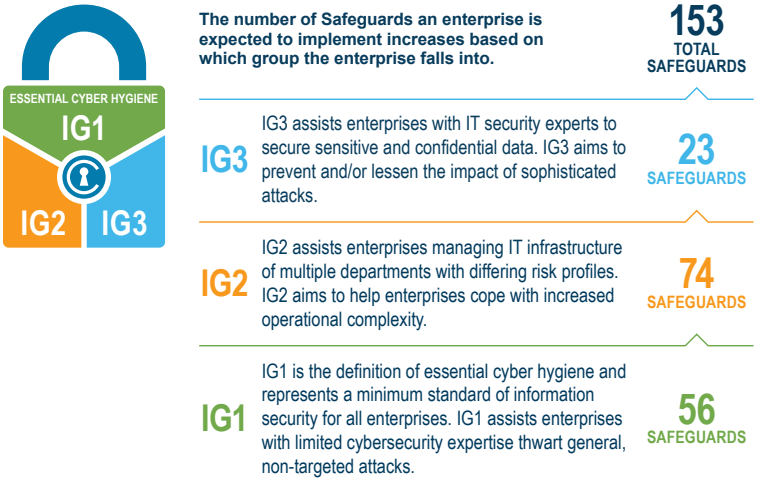

Figure 1: CIS Controls v8.1 Implementation Group levels.

IG1

An IG1 enterprise is small to medium-sized with limited IT and cybersecurity expertise to dedicate toward protecting IT assets and personnel. The principal concern of these enterprises is to keep the business operational, as they have a limited tolerance for downtime. The sensitivity of the data that they are trying to protect is low and principally surrounds employee and financial information. Safeguards selected for IG1 should be implementable with limited cybersecurity expertise and aimed to thwart general, non-targeted attacks. These Safeguards will also typically be designed to work in conjunction with small or home office commercial off-the-shelf (COTS) hardware and software.

IG2

An IG2 enterprise employs individuals responsible for managing and protecting IT infrastructure. These enterprises support multiple departments with differing risk profiles based on job function and mission. Small enterprise units may have regulatory compliance burdens. IG2 enterprises often store and process sensitive client or enterprise information and can withstand short interruptions of service. A major concern is loss of public confidence if a breach occurs. Safeguards selected for IG2 help security teams cope with increased operational complexity. Some Safeguards will depend on enterprise-grade technology and specialized expertise to properly install and configure.

IG3

An IG3 enterprise employs security experts that specialize in the different facets of cybersecurity (e.g., risk management, penetration testing, application security). IG3 assets and data contain sensitive information or functions that are subject to regulatory and compliance oversight. An IG3 enterprise must address availability of services and the confidentiality and integrity of sensitive data. Successful attacks can cause significant harm to the public welfare. Safeguards selected for IG3 must abate targeted attacks from a sophisticated adversary and reduce the impact of zero-day attacks.

If you would like to know more about how the CIS Controls and Implementation Groups pertain to enterprises of all sizes, visit our website at https://www.cisecurity.org/controls/cis-controls-list/.

Appendix B: Acronyms and Abbreviations

AAA	Authentication, Authorization, and Accounting
ABAC	Attribute-Based Access Control
ACL	Access Control List
AI	Artificial Intelligence
API	Application Programming Interface
ASLR	Address Space Layout Randomization
BEC	Business Email Compromise
BOM	Bill of Materials
C2	Command and Control
CDR	Content Disarm and Reconstruction
CI/CD	Continuous Integration/Continuous Delivery
CIS	Center for Internet Security
CMDB	Configuration Management Database
CNI	Container Network Interface
CSP	Cloud Service Provider
CVE	Common Vulnerabilities and Exposures
DAST	Dynamic Application Security Testing
DB	Database
DEP	Data Execution Prevention
DHCP	Dynamic Host Configuration Protocol
DKIM	DomainKeys Identified Mail
DLP	Data Loss Prevention
DMARC	Domain-based Message Authentication, Reporting, and Conformance
DNS	Domain Name System
EDR	Endpoint Detection and Response
gRPC	gRPC Remote Procedure Call
HIDS	Host-based Intrusion Detection System
HIPS	Host-based Intrusion Prevention System
HTML	HyperText Markup Language
HTTP/S	Hypertext Transfer Protocol (Secure)
IAM	Identity and Access Management
IDE	Integrated Development Environment
IdP	Identity Provider
IDS	Intrusion Detection System
IG	Implementation Group
IP	Internet Protocol
IPS	Intrusion Prevention System
IR	Incident Response
ISAC	Information Sharing and Analysis Center
IT	Information Technology
JSON	JavaScript Object Notation
LDAP	Lightweight Directory Access Protocol
LLM	Large Language Models
MCP	Model Context Protocol
MDM	Mobile Device Management
MFA	Multi-Factor Authentication
MIME	Multipurpose Internet Mail Extension
NAC	Network Access Control
NIDS	Network Intrusion Detection System
NIPS	Network Intrusion Prevention System
OS	Operating System
OWASP	Open Worldwide Application Security Project
PCI DSS	Payment Card Industry Data Security Standard
PII	Personally Identifiable Information
RAG	Retrieval-Augmented Generation
RBAC	Role-Based Access Control
REST	Representational State Transfer
SAST	Static Application Security Testing
SDK	Software Development Kit
SDLC	Software Development Life Cycle
SIEM	Security Information and Event Management
SIP	System Integrity Protection
SLA	Service Level Agreement
SLM	Small Language Model
SOC 2	System and Organization Controls 2
SPF	Sender Policy Framework
SSH	Secure Shell
SSO	Single Sign-On
SWG	Secure Web Gateway
TLS	Transport Layer Security
URL	Uniform Resource Locator
VLAN	Virtual Local Area Network
VM	Virtual Machine
VPC	Virtual Private Cloud
VPN	Virtual Private Network
WDEG	Windows Defender Exploit Guard
XSS	Cross-Site Scripting
YAML	YAML Ain't Markup Language

Appendix C: Links and Resources

- **CIS Critical Security Controls (CIS Controls) v8.1:** Learn more about the CIS Controls, including how to get started, why each Control is critical, procedures and tools to use during implementation, and a complete listing of Safeguards for each Control.
- **CIS Controls Policy Templates:** Policy templates geared toward Safeguards found in IG1 of the CIS Controls.
- **A Roadmap to the CIS Controls:** There is a broader ecosystem that surrounds the CIS Controls that offers guidance, tools, resources, mappings, and more to help facilitate the adoption and implementation of the framework. This guide will help adopters understand what is available to them, where to start, and how to put it all together.
- **Establishing Essential Cyber Hygiene:** IG1 is essential cyber hygiene and represents a minimum standard of information security for all enterprises. This guide will help enterprises establish essential cyber hygiene..
- **Guide to Asset Classes:** In v8.1, CIS restructured Asset Classes and their respective definitions to ensure consistency throughout the Controls. Learn more about our naming conventions and what they mean..
- **Guide to Implementation Groups (IG):** IGs are the recommended guidance to prioritize implementation of the CIS Controls. In an effort to assist enterprises of every size, IGs are divided into three groups. Learn more about the five factors that can impact IG selection for an enterprise.
- **CIS Controls Assessment Specification:** Provides an understanding of what should be measured in order to verify that the Safeguards are properly implemented.
- **CIS Controls Navigator:** Learn more about the Controls and Safeguards and see how they map to other security standards (e.g., CMMC, NIST SP 800-53 Rev. 5, PCI DSS, MITRE ATT&CK). Available for CIS Controls versions 8.1, 8, and 7.1.
- **CIS Community Defense Model (CDM) v2.0:** A guide published by CIS that leverages the open availability of comprehensive summaries of attacks and security incidents, and the industry-endorsed ecosystem that is developing around the MITRE ATT&CK Framework.
- **CIS Risk Assessment Method (CIS RAM) v2.2:** An information security risk assessment method that helps enterprises implement and assess their security posture against the CIS Controls.
- **CIS SecureSuite® Membership:** Membership with access to CIS-CAT Pro Assessor, CIS Build Kits, CIS Benchmarks, and more.
- **CIS Benchmarks®:** Secure configuration guidelines for 100+ technologies, including operating systems, applications, and network devices.
- **CIS SecureSuite Platform:** A unified platform for CIS SecureSuite Members that provides organizations with the ability to assess their cybersecurity posture against the CIS Critical Security Controls® (CIS Controls®) and to demonstrate conformance with the CIS Benchmarks®.
- **CIS Build Kits:** ZIP files that contain a Group Policy Object (GPO) for each profile within the corresponding CIS Benchmark.
- **CIS Hardened Images®:** Virtual machine images securely pre-configured to the CIS Benchmarks.
- **CIS WorkBench:** Get involved in one of our many communities
- **CIS Password Policy Guide:** CIS guidance for secure usage of passwords in an enterprise.

The Center for Internet Security, Inc. (CIS®) makes the connected world a safer place for people, businesses, and governments through our core competencies of collaboration and innovation. We are a community-driven nonprofit, responsible for the CIS Critical Security Controls® and CIS Benchmarks®, globally recognized best practices for securing IT systems and data. We lead a global community of IT professionals to continuously evolve these standards and provide products and services to proactively safeguard against emerging threats. Our CIS Hardened Images® provide secure, on-demand, scalable computing environments in the cloud. CIS is home to the Multi-State Information Sharing and Analysis Center® (MS-ISAC®), the trusted resource for cyber threat prevention, protection, response, and recovery for U.S. State, Local, Tribal, and Territorial government entities, and the Elections Infrastructure Information Sharing and Analysis Center® (EI-ISAC®), which supports the rapidly changing cybersecurity needs of U.S. election offices. To learn more, visit http://cisecurity.org or follow us on X: @CISecurity.

cisecurity.org
email@cisecurity.org
518-266-3460
Center for Internet Security
@CISecurity
CenterforIntSec
cisecurity
TheCISecurity

CIS Center for Internet Security®

CIS Controls®